Lecture 7. Diagonalization and Alternation

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Last lecture

- PATH is NL-Complete
- PATH is in SPACE($\log^2 n$)
- Savitch's theorem: NSPACE($S(n)$) $\subseteq$ SPACE($S(n)^2$)
- Immerman's theorem: NL=coNL
- TQBF is PSPACE-Complete



Back to undecidability of A_{TM}

- We write a procedure D with input <M>:
  - If H can decide A_{TM} = {<M,w> | M is a TM and M accepts w}

```
D(<M>):
        (<M> is the description of a TM)
        If H(<M,<M>>) accepts
                then reject;
                else accept;
```

- Consider D(<D>),



- We write a procedure D with input <M>:
  - If H can decide A_{TM} = {<M,w> | M is a TM and M accepts w}

```
D(<M>):
        (<M> is the description of a TM)
        If H(<M,<M>>) accepts
                then reject;
                else accept;
```

- Consider D(<D>),
  - if D(<D>) accepts, H(<D,<D>>) will accept, then contradicts



- We write a procedure D with input <M>:
  - If H can decide A_{TM} = {<M,w> | M is a TM and M accepts w}

```
D(<M>):
        (<M> is the description of a TM)
        If H(<M,<M>>) accepts
                then reject;
                else accept;
```

- Consider D(<D>),
  - if D(<D>) rejects, H(<D,<D>>) will reject, then contradicts



Back to undecidability of A_{TM}

- What if:

```
D(<M>):
        (<M> is the description of a TM)
        Simulate M with input <M> for |<M>|^|.| steps;
        if accepts then reject;
                   else accept;
```

- What does it mean?



Back to undecidability of A_{TM}.

- What if:

```
D(<M>):
        (<M> is the description of a TM)
        Simulate M with input <M> for |<M>|^1.1 steps;
        if accepts then reject;
                   else accept;
```

- The language L(D) of machine D is in TIME($n^{1.1}$)
  - since D always halts in $O(n^{1.1})$ time
  - Is there a contradiction when considering D(<D>)?



Back to undecidability of A_{TM}.

- What if:

D(<D>) ≠ D(<D>) ?

```
D(<D>):
        Simulate D(<D>) for |<D>|^1.1 steps;
        if accepts then reject;
                else accept;
```

Simulation takes a little bit more:

If M(x) takes f(|x|) time, to simulate M on a universal TM takes $c \cdot f(|x|) \cdot \log f(|x|)$ steps

Here D takes $n^{1.1}$ time, but maybe we cannot finish the simulation of D(<D>) in $n^{1.1}$ steps

So we will not get a contradiction for D(<D>)



L(D) is not in TIME(n)

- Proof:
  - If L(D) is in TIME(n), that is, there is a machine M' deciding L(D) in c·n time
  - *That is, for all x, M'(x)=D(x)*
  - Then consider D(<M'>)=M'(<M'>)

```
D(<M'>):
        Simulate M' with input <M'> for |<M'>|^1.1 steps;
        if accepts then reject;
                else accept;
```



L(D) is not in TIME(n)

- Proof:
  - If L(D) is in TIME(n), that is, there is a machine M' deciding L(D) in c·n time.
  - *That is, for all x, M'(x)=D(x)*
  - we can make <M'> not too small, then $|\langle M' \rangle|^{1.1} > c \cdot |\langle M' \rangle| \cdot \log(|\langle M' \rangle|)$, so the simulation can finish
  - contradiction to D(<M'>)=M'(<M'>)

```
D(<M'>):
        Simulate M' with input <M'> for |<M'>|^1.1 steps;
        if accepts then reject;
                   else accept;
```



Conclusion

D(<M>):
(<M> is the description of a TM)
Simulate M with input <M> for $|<M>|^{1.1}$ steps;
if accepts then reject;
else accept;

- The language L(D) of machine D is in $TIME(n^{1.1})$
- But cannot be decided in TIME(n)



Time Hierarchy Theorem

Theorem

For any *time constructible* function $f\colon \mathbb{N} \to \mathbb{N}$, there exists a language $\mathcal{L}$ that is decidable in time $O(f(n))$, but not in time $o\left(\frac{f(n)}{\log f(n)}\right)$



Space Hierarchy Theorem *(proof omitted)*

Theorem

For any space constructible function $f: \mathbb{N} \to \mathbb{N}$, there exists a language $\mathcal{L}$ that is decidable in space $O(f(n))$, but not in space $o(f(n))$

Corollary

If $f(n)$ and $g(n)$ are space constructible functions, and $f(n) = o(g(n))$, then $\text{SPACE}[f] \subsetneq \text{SPACE}[g]$



Hierarchy theorems

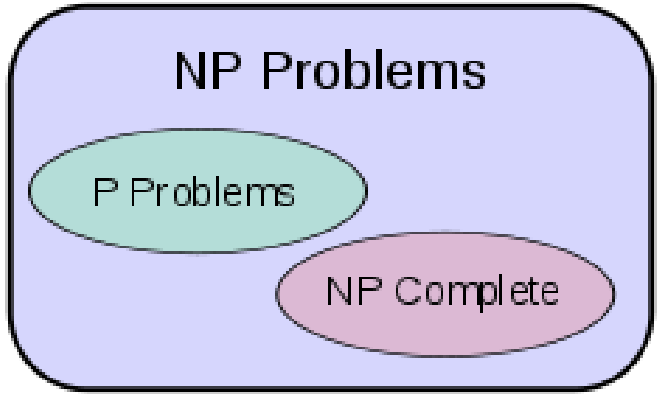
- We have known $L \subseteq NL \subseteq P \subseteq NP \subseteq PSPACE$
  - But we don't know whether they are proper subsets

- What about L and PSPACE?
- Space hierarchy theorem: $L \neq PSPACE$
  - L is a proper subset of PSPACE
  - More time/space gives us more power



Ladner's theorem

- In NP, most problems are either in P or in NP-Complete





Ladner's theorem

- In NP, most problems are either in P or in NP-Complete

- P
  - Connectivity
  - Shortest path
  - Maximum flow
  - Maximum matching
  - Linear programming
  - Primality
  - Eulerian path
  - Edit distance
  - Planarity
  - …

- NP-Complete
  - Boolean satisfiability problem (Sat.)
  - Knapsack problem
  - Hamiltonian path
  - Travelling salesman problem
  - Subgraph isomorphism
  - Subset sum
  - Clique problem
  - Vertex cover
  - Independent set
  - Graph coloring
  - …



Ladner's theorem

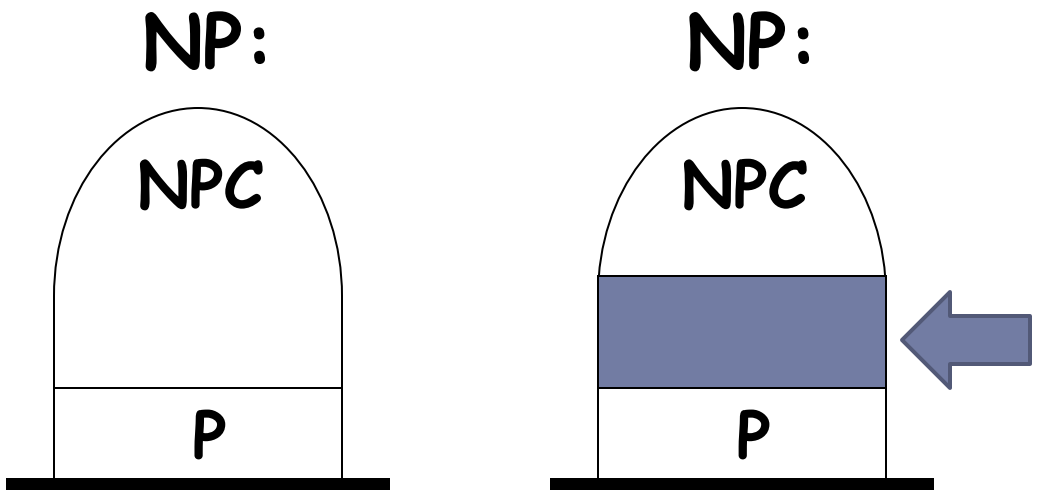
- In NP, most problems are either in P or in NP-Complete
- Also, there are some problems we don't know whether they are in P or in NP-Complete:
  - Nash equilibrium
  - Graph isomorphism
  - Integer factoring
  - Parity game
- Bold conjecture: every problem of NP is either in P or NP-Complete?





Ladner's theorem

- *[Ladner 1975]* Suppose that P≠NP, then there exists NP-intermediate problems (in NP\P and not NP-Complete)
  - Proof: by "lazy diagonalization"





Ladner's Theorem

- Define M_i to be the TM with code i
- Add a clock to M_i so that it runs in at most n^i steps
  - If it doesn't terminate after n^i steps, just reject

- So all M_i are in polynomial time
  - So we can enumerate (TMs deciding) all languages in **P**
  - By enumerating all polynomial time machines



Ladner's Theorem

- Similarly, we can also enumerate all **NP**-hard languages
  - enumerate TMs f_i computing all polynomial-time functions in n^i steps
  - machine N_i decides the language which SAT reduces to via f_i *if f_i is a reduction,*
  - (N_i virtually exists if f_i is a reduction, but we don't need to construct it)



Ladner's Theorem

- Similarly, we can also enumerate all **NP**-hard languages.
  - enumerate TMs f_i computing all polynomial-time functions in n^i steps
  - machine N_i decides the language which SAT reduces to via f_i *if f_i is a reduction,*
  - (N_i virtually exists if f_i is a reduction, but we don't need to construct it)

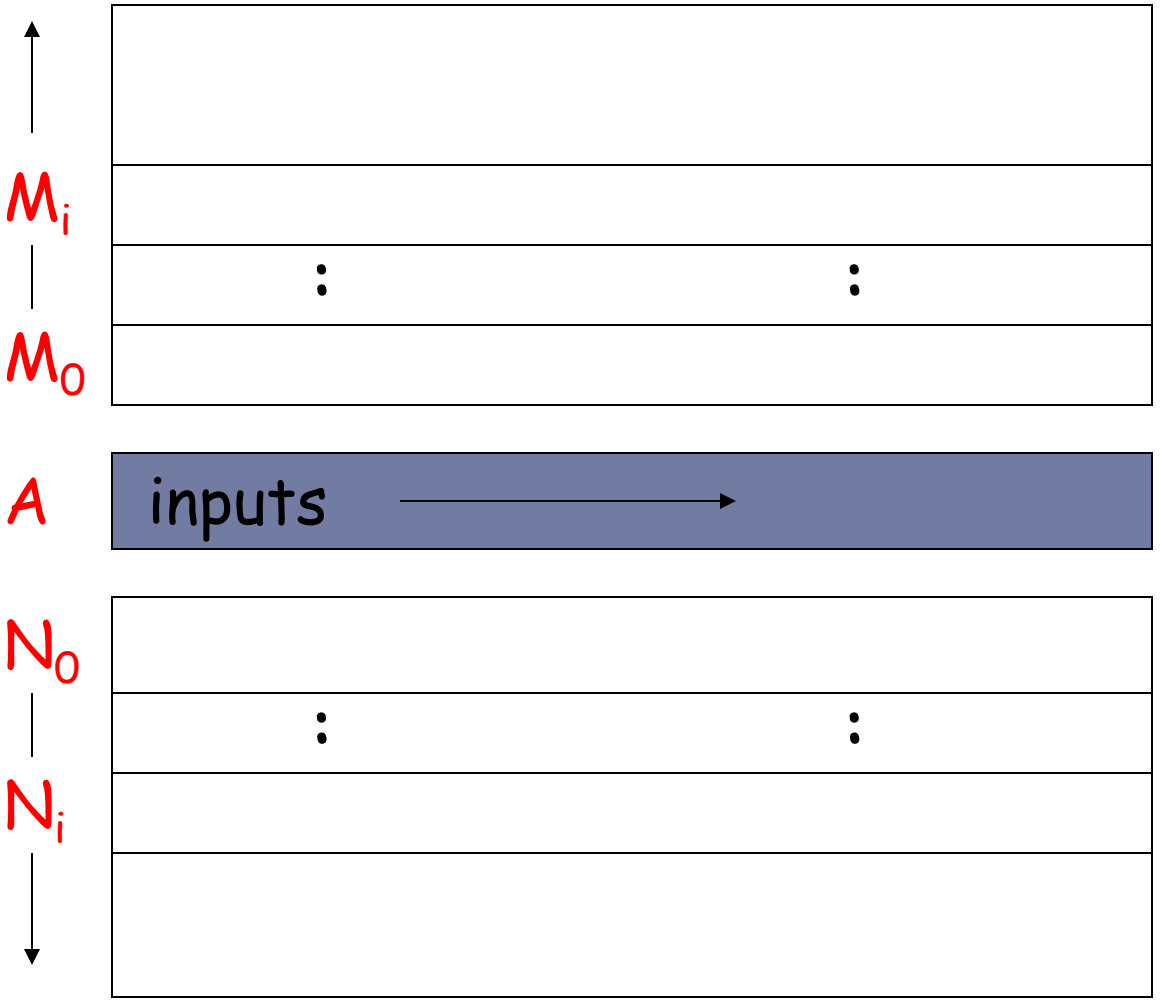
$$x \in SAT \Leftrightarrow f_i(x) \in L(N_i)$$

If $P \neq NP$, $L(N_i)$ is not in P



Ladner's Theorem

- Our goal: construct A ∈**NP** that is neither in **P** nor **NP**-hard



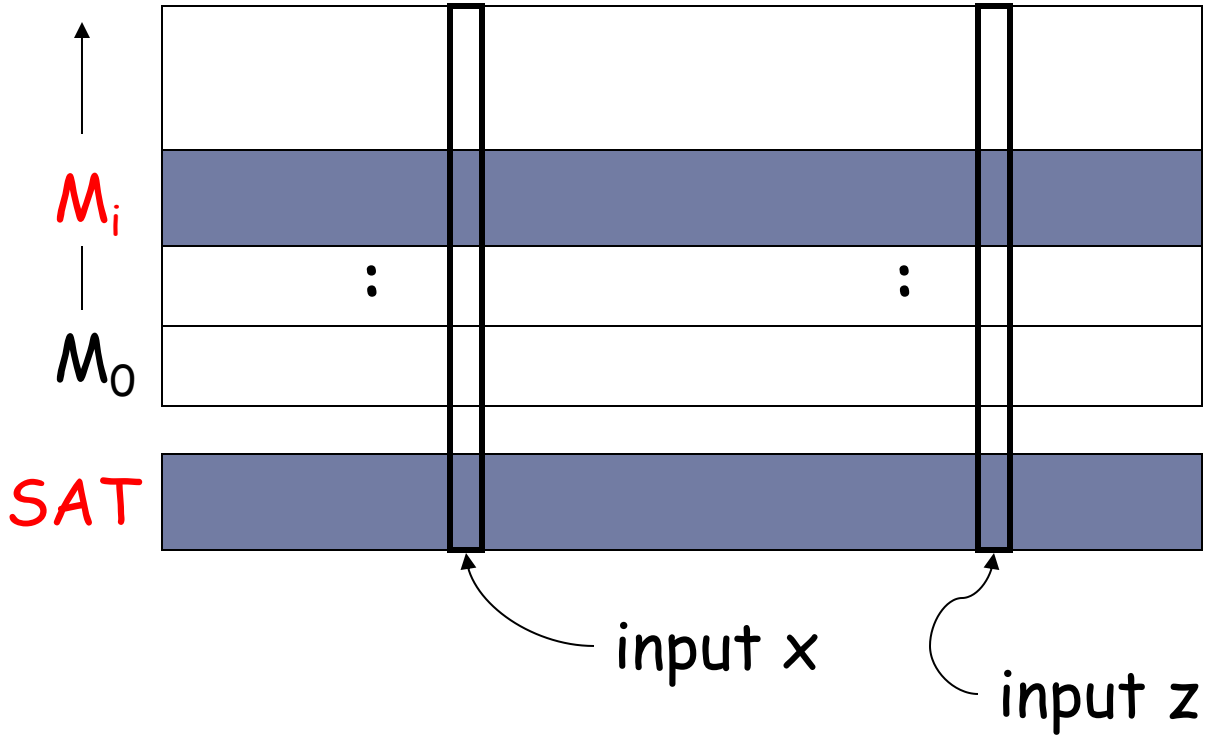


Ladner's Theorem

- Top half, assuming **P ≠ NP:**
  - M_i are polynomial time machines

• focus on M_i

• for any x, we can always find some z ≥ x on which M_i and SAT differ (why?)





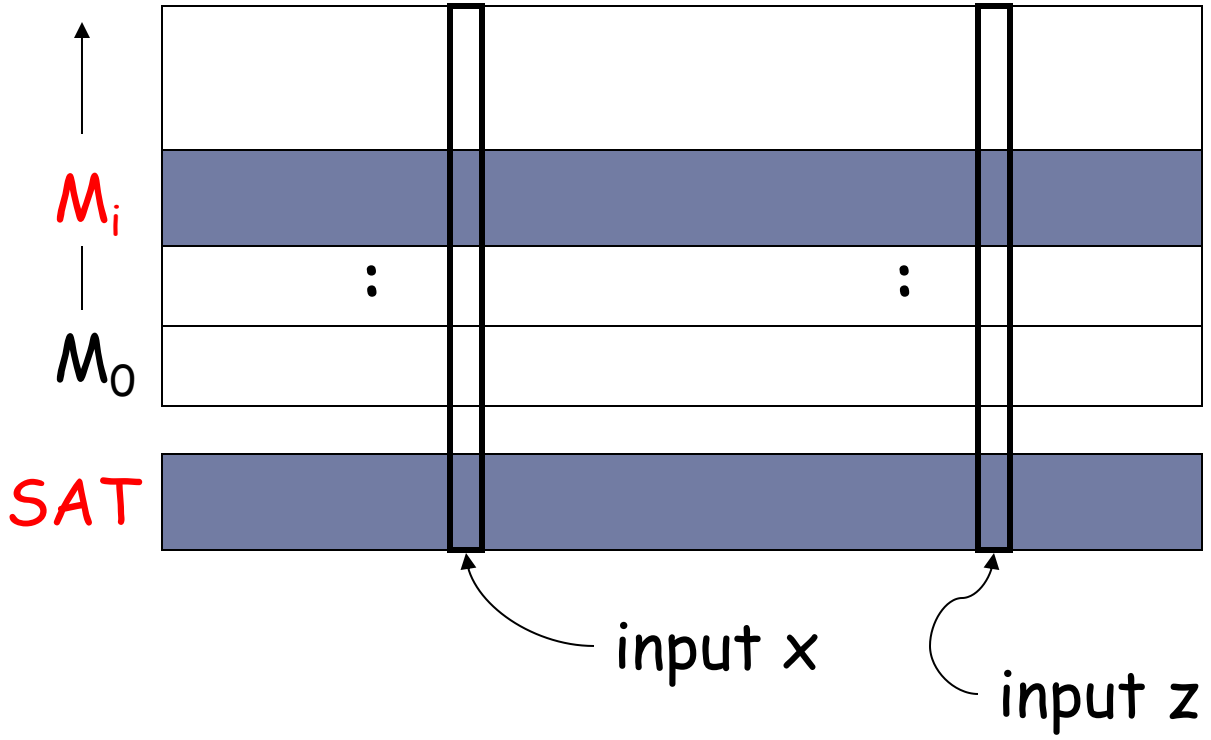
Ladner's Theorem

- Top half, assuming **P ≠ NP:**
  - M_i are polynomial time machines

• focus on M_i

• for any x, we can always find some z ≥ x on which M_i and SAT differ

•otherwise SAT is in **P**



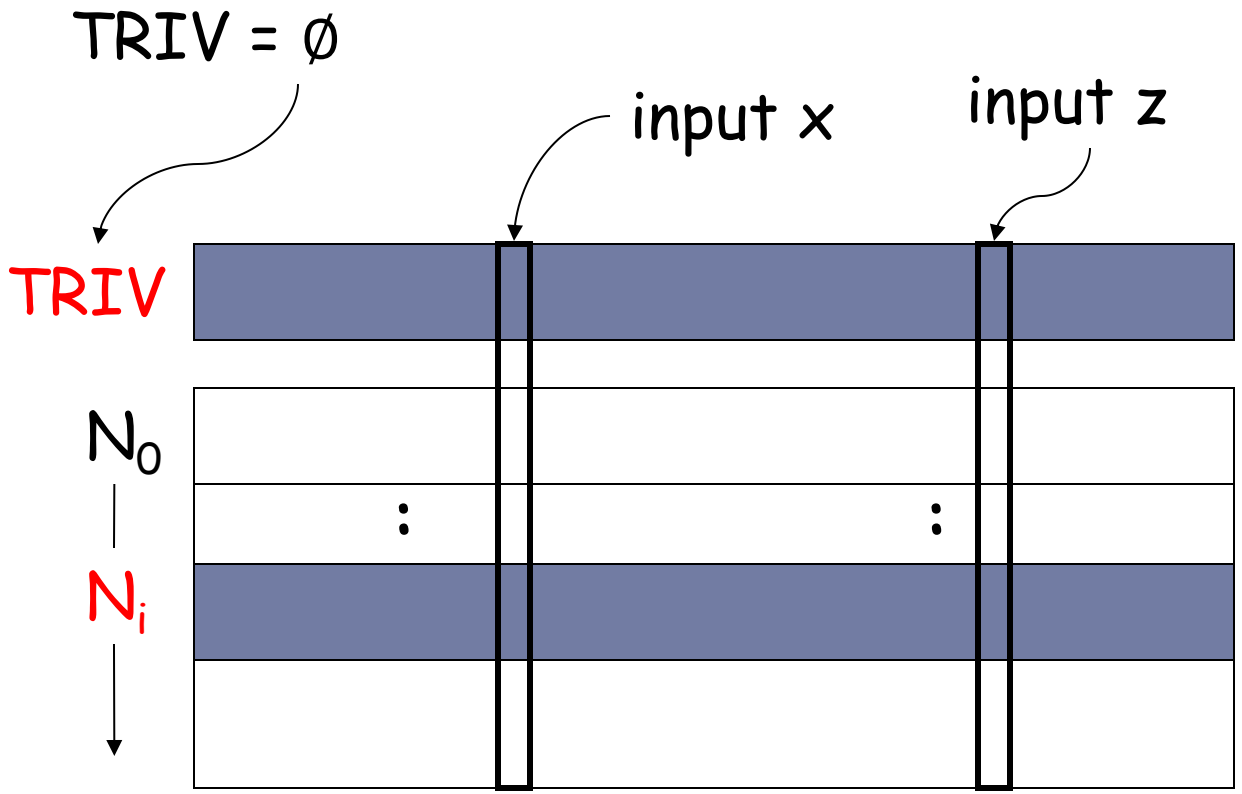


Ladner's Theorem

- Bottom half, assuming **P ≠ NP:**
  - N_i are machines determining NP-Complete problems (ideally)

• focus on N_i

• for any x, can always find some $z \geq x$ on which N_i and TRIV differ



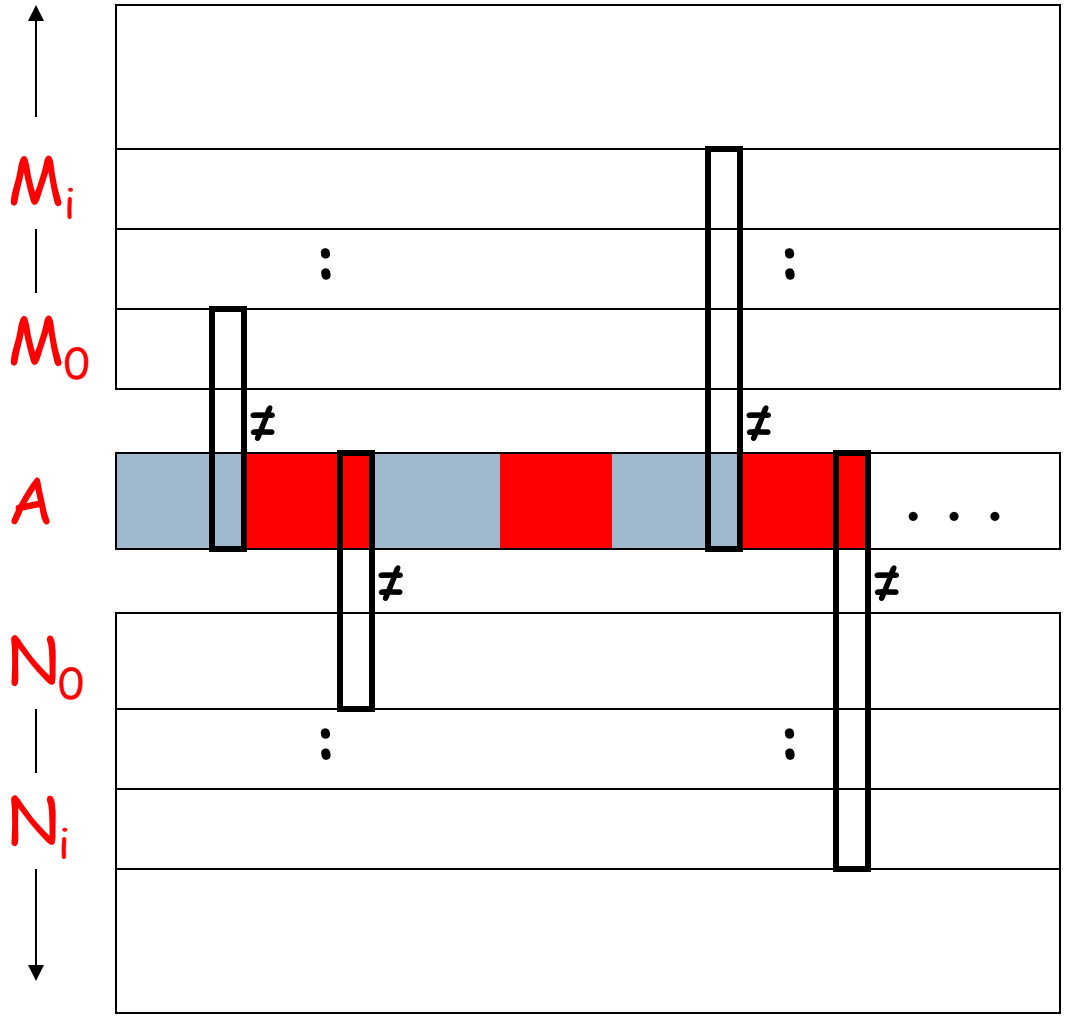


Ladner's Theorem

- Try to "merge":

SAT TRIV

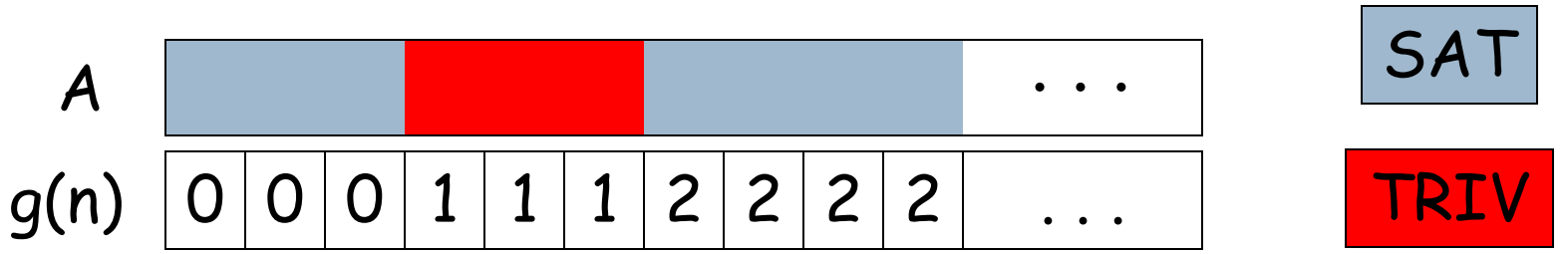
- on input x, either
  - A(x)=SAT(x)
  - A(x)=TRIV(x)
- if we can decide which one in polynomial time, A∈**NP**





Ladner's Theorem

- General scheme: g(n) slowly increasing function
  - n: length of the input x
  - g(n) even: answer SAT(x)
  - g(n) odd: answer TRIV(x)



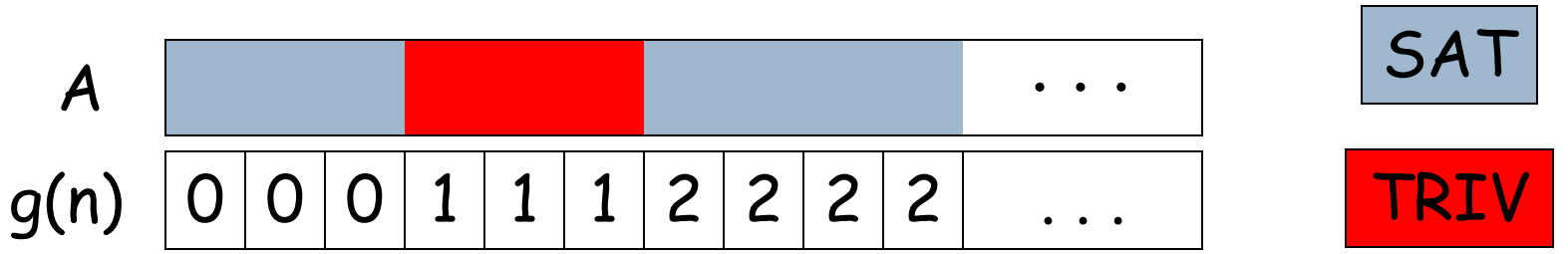
Our aim: all M_i and N_i doesn't decide A and A∈**NP**



Ladner's Theorem

- General scheme: g(n) slowly increasing function

  $A=\{x \mid x \in SAT \text{ and } g(|x|) \text{ is even}\}$



Our aim: all M_i and N_i doesn't decide A and A∈**NP**



Ladner's Theorem

- 1st attempt to define g(n)
- "eager g(n)": increase at 1st opportunity
- Inductive definition: g(0) = 0; g(n) =
  - if g(n-1) = 2i, trying to kill M_i
    - if $\exists z, |z|=n-1$ s.t. $M_i(z) \neq SAT(z)$, then g(n) = g(n-1) + 1; else g(n) = g(n-1)

Our aim: all M_i and N_i doesn't decide L



Ladner's Theorem

- 1st attempt to define g(n)
- "eager g(n)": increase at 1st opportunity
- Inductive definition: g(0) = 0; g(n) =
  - if g(n-1) = 2i, trying to kill M_i
    - if $\exists z, |z|=n-1$ s.t. $M_i(z) \neq SAT(z)$, then g(n) = g(n-1) + 1; else g(n) = g(n-1)
  - if g(n-1) = 2i+1, trying to kill N_i
    - if $\exists z, |z|=n-1$ s.t. $N_i(z) \neq TRIV(z)$, then g(n) = g(n-1) + 1; else g(n) = g(n-1)

Our aim: all M_i and N_i doesn't decide L



Ladner's Theorem

- 1st attempt to define g(n)
- "eager g(n)": increase at 1st opportunity
- Inductive definition: g(0) = 0; g(n) =
  - if g(n-1) = 2i, trying to kill M_i
    - if $\exists z, |z|=n-1$ s.t. $M_i(z) \neq SAT(z)$, then g(n) = g(n-1) + 1; else g(n) = g(n-1)
  - if g(n-1) = 2i+1, trying to kill N_i
    - if $\exists z, |z|=n-1$ s.t. $N_i(z)=1$, then g(n) = g(n-1) + 1; else g(n) = g(n-1)

hard to compute N_i

Our aim: all M_i and N_i doesn't decide L



Ladner's Theorem

- Problem: eager g(n) too difficult to compute
- on input of length n,
  - look at all strings z of length n-1
  - compute SAT(z) for each, time needed: $2^{O(n)}$
- Solution:
  - To compute g(n), only looks for strings z of length <loglog n



Ladner's Theorem

$A=\{x \mid x \in SAT \text{ and } g(|x|) \text{ is even}\}$

- Let $g(0)=g(1)=2$. If $\log^{g(n)} n \geq n$ then let $g(n+1)=g(n)$, otherwise:
  - if $g(n) = 2i$:
    - find z s.t. $|z|<\log\log n$, $g(|z|)$ even and $SAT(z) \neq M_i(z)$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$



Ladner's Theorem

$A=\{x \mid x \in SAT \text{ and } g(|x|) \text{ is even}\}$

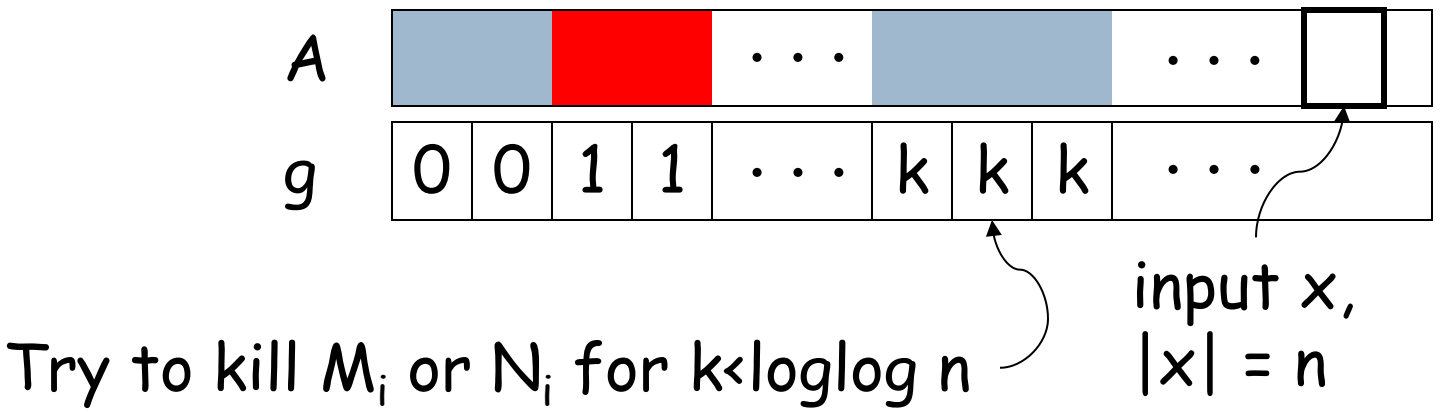
- Let $g(0)=g(1)=2$. If $\log^{g(n)} n \geq n$ then let $g(n+1)=g(n)$, otherwise:
  - if $g(n) = 2i$:
    - find z s.t. $|z|<\log\log n$, $g(|z|)$ even and $SAT(z) \neq M_i(z)$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$
  - if $g(n) = 2i + 1$:
    - find z such that $N_i(z) \neq A(z)$
    - if f_i is a reduction, $x \in SAT \Leftrightarrow f_i(x) \in L(N_i)$
    - find x s.t. $|x|<\log\log n$, either
      - $g(|f_i(x)|)$ is odd and $x \in SAT$, so $f_i(x) \in L(N_i)$ but $f_i(x) \notin A$
      - $g(|f_i(x)|)$ is even and $SAT(x) \neq SAT(f_i(x))$, so $N_i(f_i(x)) \neq A(f_i(x))$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$

So we can compute g(n) in time polynomial in n



Ladner's Theorem

- Inductive definition of g(n)





Ladner's Theorem

$A=\{x \mid x \in SAT \text{ and } g(|x|) \text{ is even}\}$

- Let $g(0)=g(1)=2$. If $\log^{g(n)} n \geq n$ then let $g(n+1)=g(n)$, otherwise:
  - if $g(n) = 2i$:
    - find z s.t. $|z|<\log\log n$, $g(|z|)$ even and $SAT(z) \neq M_i(z)$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$
  - if $g(n) = 2i + 1$:
    - find z such that $N_i(z) \neq A(z)$
    - if f_i is a reduction, $x \in SAT \Leftrightarrow f_i(x) \in L(N_i)$
    - find x s.t. $|x|<\log\log n$, either
      - $g(|f_i(x)|)$ is odd and $x \in SAT$, so $f_i(x) \in L(N_i)$ but $f_i(x) \notin A$
      - $g(|f_i(x)|)$ is even and $SAT(x) \neq SAT(f_i(x))$, so $N_i(f_i(x)) \neq A(f_i(x))$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$

If g gets stuck at 2i, then $SAT(z)=M_i(z)$ for $z : |z| > n_0$, which implies SAT $\in$ **P**. Contradiction.



Ladner's Theorem

$A=\{x \mid x \in SAT \text{ and } g(|x|) \text{ is even}\}$

- Let $g(0)=g(1)=2$. If $\log^{g(n)} n \geq n$ then let $g(n+1)=g(n)$, otherwise:
  - if $g(n) = 2i$:
    - find z s.t. $|z|<\log\log n$, $g(|z|)$ even and $SAT(z) \neq M_i(z)$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$
  - if $g(n) = 2i + 1$:
    - find z such that $N_i(z) \neq A(z)$
    - if f_i is a reduction, $x \in SAT \Leftrightarrow f_i(x) \in L(N_i)$
    - find x s.t. $|x|<\log\log n$, either
      - $g(|f_i(x)|)$ is odd and $x \in SAT$, so $f_i(x) \in L(N_i)$ but $f_i(x) \notin A$
      - $g(|f_i(x)|)$ is even and $SAT(x) \neq SAT(f_i(x))$, so $N_i(f_i(x)) \neq A(f_i(x))$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$

If g gets stuck at 2i+1
When f_i is a reduction, $f_i(x)$ keeps growing and $g(|f_i(x)|)$ will get 2i+1, then there must exist $x \notin SAT$.
When $g(|f_i(x)|)$ is even, $|f_i(x)|$ must be bounded, so $SAT(x)=SAT(f_i(x))$ can be computed in constant time. So if g gets stuck at odd number, we can decide $x \in SAT$ in **P**.



Ladner's Theorem

$A=\{x \mid x \in SAT \text{ and } g(|x|) \text{ is even}\}$

- Let $g(0)=g(1)=2$. If $\log^{g(n)} n \geq n$ then let $g(n+1)=g(n)$, otherwise:
  - if $g(n) = 2i$:
    - find z s.t. $|z|<\text{loglog } n$, $g(|z|)$ even and $SAT(z) \neq M_i(z)$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$
  - if $g(n) = 2i + 1$:
    - find z such that $N_i(z) \neq A(z)$
    - if f_i is a reduction, $x \in SAT \iff f_i(x) \in L(N_i)$
    - find x s.t. $|x|<\text{loglog } n$, either
      - $g(|f_i(x)|)$ is odd and $x \in SAT$, so $f_i(x) \in L(N_i)$ but $f_i(x) \notin A$
      - $g(|f_i(x)|)$ is even and $SAT(x) \neq SAT(f_i(x))$, so $N_i(f_i(x)) \neq A(f_i(x))$
    - if found, $g(n+1) = g(n)+1$ else $g(n+1)=g(n)$

$|f_i(x)| \leq |x|^i < |x|^{g(n)} < \text{loglog}^{g(n)} n < \log n$



Ladner's Theorem

- Finishing up:

$$A = \{ x \mid x \in SAT \text{ and } g(|x|) \text{ even} \}$$

- $A \in$ **NP** since $g(|x|)$ can be computed in $poly(|x|)$ time



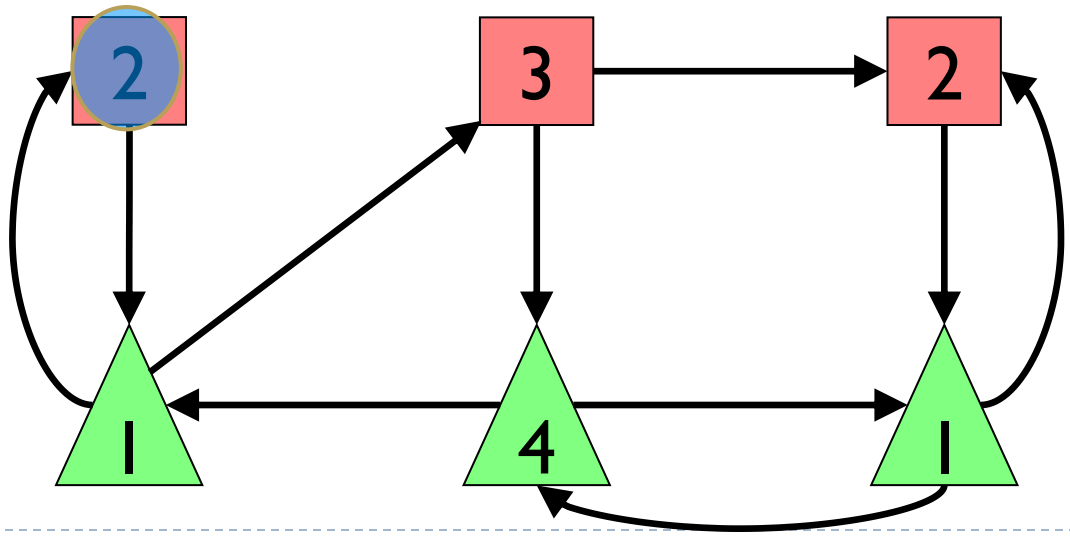
NP-intermediate problems

- Proofs by diagonalization usually don't give any natural problems
- If SAT$\in$P, we can also find the satisfying assignment in polynomial time
  - For NP-Complete problems, Decision problem$\Leftrightarrow$ Find a solution
  - But for integer factoring and Nash equilibrium it is not the case
  - Integer factoring: primality (whether it can be factored) is in P
  - Nash equilibrium always exists



Parity Game

- In a graph G=($V_0 \cup V_1$, E, p), Player 0 and Player 1 play a game with a token:
- a "play" is then an infinite sequence of nodes $u_0, u_1, u_2, \ldots$
  - Player 0 wins if the largest priority $p(u_i)$ seen infinite often is EVEN
  - Player 1 wins if the largest priority $p(u_i)$ seen infinite often is ODD





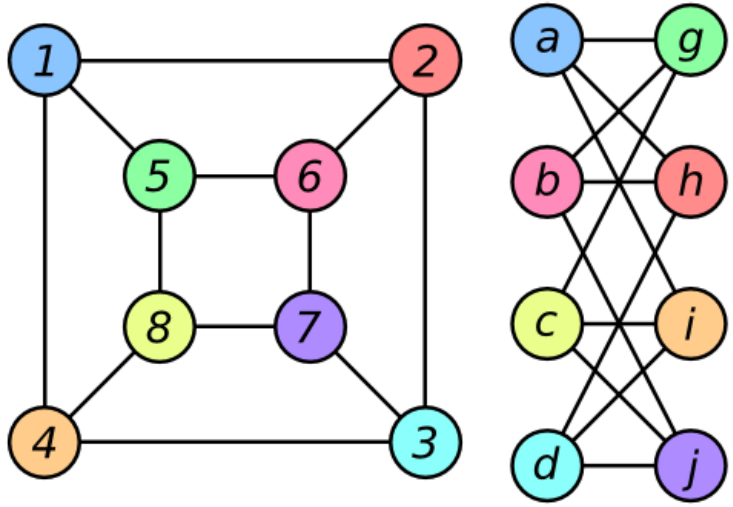
Two-player game in NP

- In a graph $G=(V_0 \cup V_1, E, p)$, Player 0 and Player 1 play a game with a token:
- a "play" is then an infinite sequence of nodes $u_0, u_1, u_2, \ldots$
  - Player 0 wins if the largest priority $p(u_i)$ seen infinite often is EVEN
  - Player 1 wins if the largest priority $p(u_i)$ seen infinite often is ODD

- If a two-player game is in NP, it is also in coNP
- If it is NP-complete, then NP=coNP

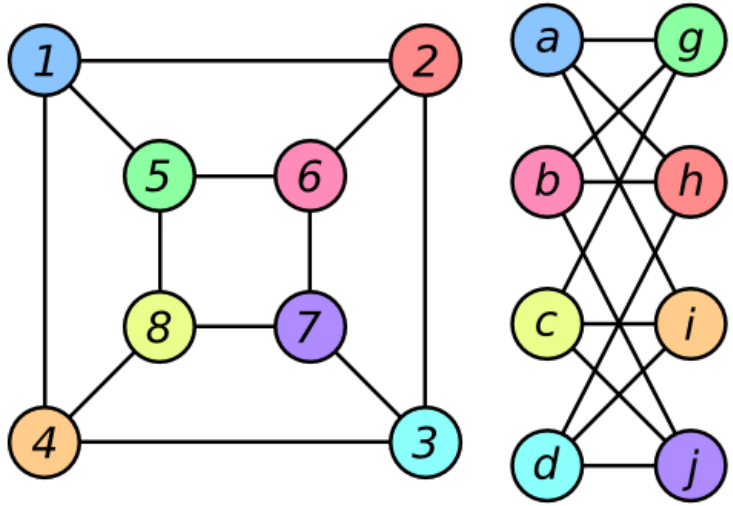


- What about Graph Isomorphism?
  - Why is it unlikely in NPC?





- What about Graph Isomorphism?
  - *[Boppana, Hastad, Zachos 1987]*
    If GI in NP-Complete, then $\sum_2^P = \prod_2^P$
  - We will define $\sum_i^P$ and $\prod_i^P$





The polynomial hierarchy PH

- For i≥1, a language L is in $\sum_i^p$ if there exists a polynomial time TM M and a polynomial q s.t.

$$x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} \forall u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

  - where Q_i is ∃ when i is odd; is ∀ when i is even

  - (sometimes we can omit superscript "p")

Theorem. Any wf. $\mathcal{A}$ is equivalent to a wf. $\mathcal{B}$ in prenex form



The polynomial hierarchy PH

- For i≥1, a language L is in $\sum_i^p$ if there exists a polynomial time TM M and a polynomial q s.t.

$$x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} \forall u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

  - where Q_i is ∃ when i is odd; is ∀ when i is even
  - $\sum_1^p$ =?



The polynomial hierarchy PH

- For i≥1, a language L is in $\sum_i^p$ if there exists a polynomial time TM M and a polynomial q s.t.

$$x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} \forall u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

  - where Q_i is ∃ when i is odd; is ∀ when i is even
  - $\sum_0^p = P$, $\sum_1^p = NP$

- Define $PH = \cup_i \sum_i^p$



The polynomial hierarchy PH

- For i≥1, a language L is in $\sum_i^p$ if there exists a polynomial time TM M and a polynomial q s.t.

$$x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} \forall u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

  - where Q_i is ∃ when i is odd; is ∀ when i is even

- Define PH=$\cup_i \sum_i^p$
- Define $\prod_i^p$ =co $\sum_i^p$, which is:

$$x \in L \Leftrightarrow \forall u_1 \in \{0,1\}^{q(|x|)} \exists u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$



The polynomial hierarchy PH

- For i≥1, a language L is in $\sum_i^p$ if there exists a polynomial time TM M and a polynomial q s.t.

$$x \in L \Leftrightarrow \exists u_1 \in \{0,1\}^{q(|x|)} \forall u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

  - where Q_i is ∃ when i is odd; is ∀ when i is even

- Define PH=$\cup_i \sum_i^p$
- Define $\prod_i^p$ =co $\sum_i^p$, which is:

$$x \in L \Leftrightarrow \forall u_1 \in \{0,1\}^{q(|x|)} \exists u_2 \in \{0,1\}^{q(|x|)} \ldots Q_i u_i \in \{0,1\}^{q(|x|)} M(x, u_1, \ldots, u_i) = 1$$

- **Note that $\sum_i^p \subseteq \prod_{i+1}^p \subseteq \sum_{i+2}^p$, so PH=$\cup_i \prod_i^p$**



- The "alternating quantifiers" definition of PH is easy to get:
  - $\sum_i^p \subseteq \prod_{i+1}^p \subseteq \sum_{i+2}^p$
  - PH⊆PSPACE

- More about this in the next lecture



Other definitions

- by Oracle Turing machines



Oracle Turing Machines

- Oracle Turing Machine (OTM):
  - multitape TM M with special "query" tape
  - special states q_{query}, q_{yes}, q_{no}
  - on input x, with oracle language A
  - M^A runs as usual, except…
  - when M^A enters state q_{query}:
    - y = contents of query tape
    - $y \in A \Rightarrow$ transition to q_{yes}
    - $y \notin A \Rightarrow$ transition to q_{no}



Oracle Turing Machines

- Nondeterministic OTM
  - defined in the same way

- oracle is like a subroutine, or function in your favorite PL
  - but each call counts as single step

    e.g.: given $\phi_1, \phi_2, \ldots, \phi_n$, are even # satisfiable?
  - poly-time OTM solves with SAT oracle



Oracle Turing Machines

Shorthand #1:

- applying oracles to entire complexity classes:
  - complexity class **C**
  - language A

$$\mathbf{C}^A = \{L \text{ decided by OTM M with oracle A with M "in" } \mathbf{C}\}$$

  - example: $\mathbf{P}^{SAT}$: polynomial time with oracle for SAT



Oracle Turing Machines

Shorthand #2:

- using complexity classes as oracles:
  - OTM M
  - complexity class **C**
  - $M^{\mathbf{C}}$ decides language L if for some language A ∈ **C**, M^A decides L

Both together: $\mathbf{C}^{\mathbf{D}}$ = languages decided by OTM "in" **C** with oracle language from **D**

e.g. $\mathbf{P}^{\mathrm{SAT}} = \mathbf{P}^{\mathbf{NP}}$



The Polynomial-Time Hierarchy

We have known

$\Sigma_0 = \Pi_0 = \mathbf{P}$

$\Sigma_1 = \mathbf{NP}$ $\Pi_1 = \mathbf{coNP}$

What about $\sum_2$ and $\prod_2$?

$\sum_2$: $x \in L \Leftrightarrow \exists u \forall v\ M(x,u,v)=1$

$\prod_2$: $x \in L \Leftrightarrow \forall u \exists v\ M(x,u,v)=1$



The Polynomial-Time Hierarchy

We have known

$\Sigma_0 = \Pi_0 = \mathbf{P}$

$\Sigma_1 = \mathbf{NP}$ $\Pi_1 = \mathbf{coNP}$

What about $\sum_2$ and $\prod_2$?

$\sum_2$: $x \in L \Leftrightarrow \exists u\ (\forall v\ M(x,u,v)=1)$

$\prod_2$: $x \in L \Leftrightarrow \forall u\ (\exists v\ M(x,u,v)=1)$

If we have an NP oracle, we can answer NP/co-NP problems immediately



The Polynomial-Time Hierarchy

We have known

$\Sigma_0 = \Pi_0 = \mathbf{P}$

$\Sigma_1 = \mathbf{NP}$ $\Pi_1 = \mathbf{coNP}$

What about $\sum_2$ and $\prod_2$?

$\sum_2$: $x \in L \Leftrightarrow \exists u\ (\forall v\ M(x,u,v)=1)$

$\prod_2$: $x \in L \Leftrightarrow \forall u\ (\exists v\ M(x,u,v)=1)$

Thus, $\Sigma_2 = \mathbf{NP}^{\mathbf{NP}}$, $\Pi_2 = \mathbf{coNP}^{\mathbf{NP}}$

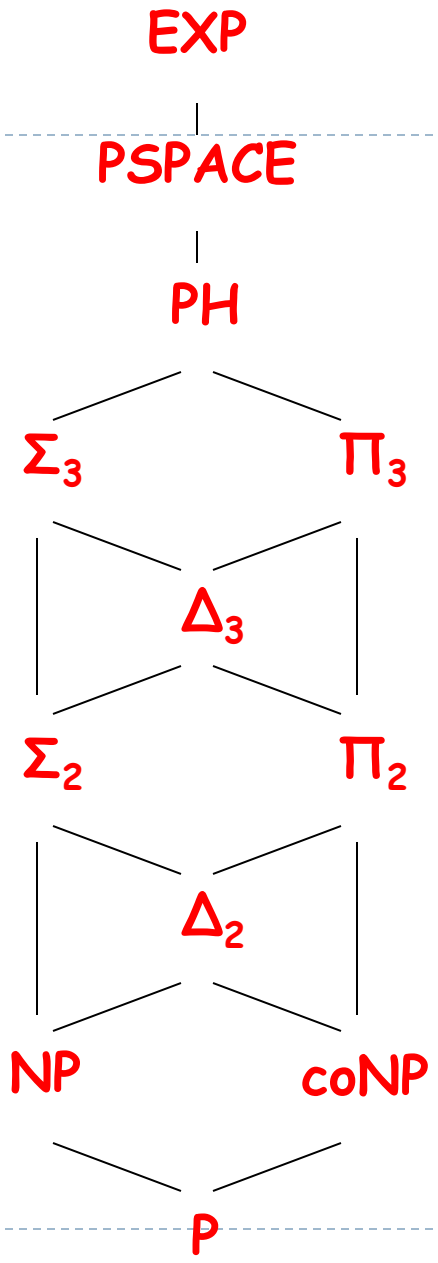


The Polynomial-Time Hierarchy

$$\Sigma_0 = \Pi_0 = P$$

$\Delta_1 = P^P$	$\Sigma_1 = NP$	$\Pi_1 = coNP$
$\Delta_2 = P^{NP}$	$\Sigma_2 = NP^{NP}$	$\Pi_2 = coNP^{NP}$
$\Delta_{i+1} = P^{\Sigma_i}$	$\Sigma_{i+1} = NP^{\Sigma_i}$	$\Pi_{i+1} = coNP^{\Sigma_i}$

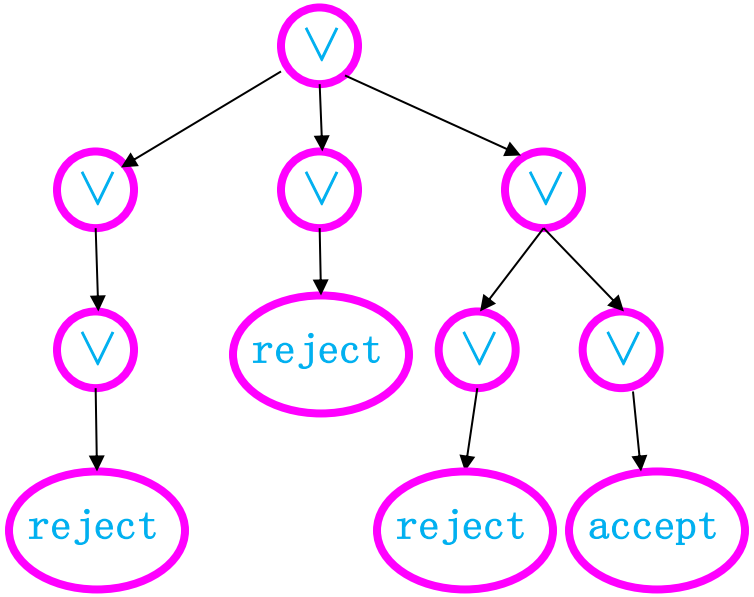
Polynomial Hierarchy $PH = \cup_i \Sigma_i$



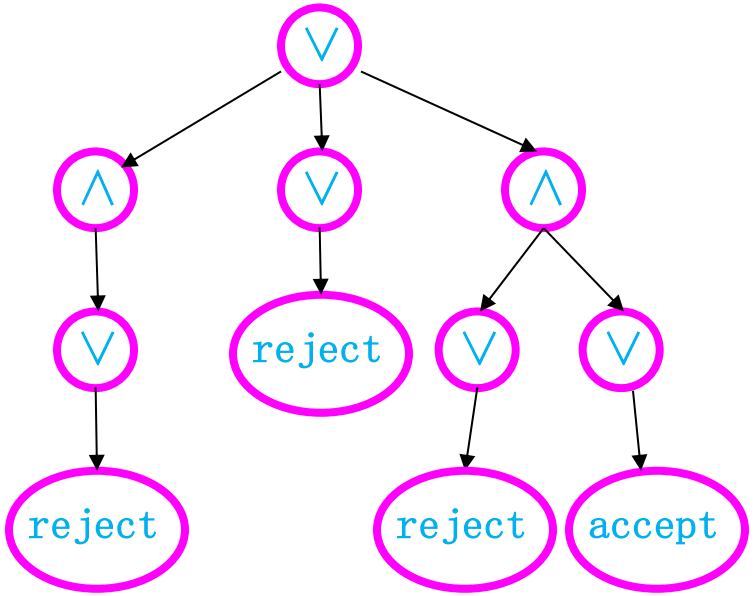
Alternations

Alternating Turing machines

Definition 10.16 An ***alternating Turing machine*** is a nondeterministic TM with an additional feature. Its states, except for the accept and reject states, are divided into ***universal states*** and ***existential states***. When we run an alternating TM on an input string, we label each node of its nondeterministic computation tree with $\wedge$ or $\vee$, depending on whether the corresponding configuration contains a universal or existential state. We determine acceptance by designating a node to be accepting if it is labeled with $\wedge$ and all of its children are accepting or if it is labeled with $\vee$ and at least one of its children is accepting.



nondeterministic computation tree



alternating computation tree

ATIME and ASPACE defined

ATIME(t(n)) $=_{def}$ {**L** | **L** is decided by an **O(t(n))** time alternating TM}

ASPACE(t(n)) $=_{def}$ {**L** | **L** is decided by an **O(t(n))** space alternating TM}

We further define ***AP***, ***APSPACE*** and ***AL*** to be the classes of languages that are decided by alternating polynomial time, alternating polynomial space, and alternating logarithmic space TMs, respectively.

Example Here is an alternating polynomial time algorithm for the UNSATISFIABILITY problem for Boolean formulas:

"On input **<φ>**:

1. Universally select all assignments to the variables of φ
2. For a particular assignment, evaluate φ
3. If φ evaluates to **0**, *accept*; otherwise *reject*.

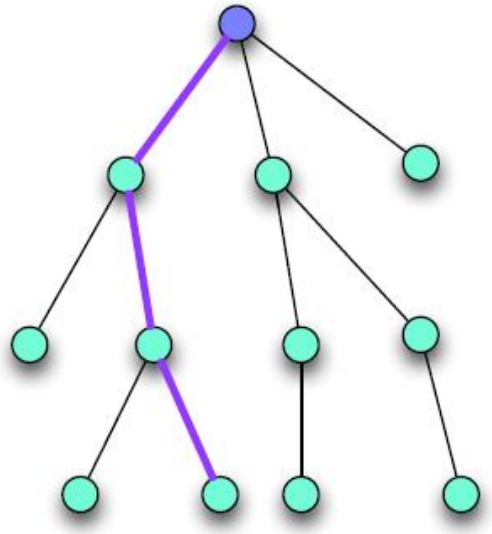


- Claim 1. ATIME(f(n)) ⊆ SPACE(?)



- Claim 1. ATIME(f(n)) ⊆ SPACE(f(n))
  - Do a DFS to explore all possible branches
  - The space is only for choices at each branch

The choices: 1,2,2





- Claim 2. SPACE(f(n)) ⊆ ATIME(?)
  - *proof similar to the PSPACE-completeness of TQBF*

- In the TM M, For configurations u,v and an integer t, define ψ(u,v,t)=true iff v is reachable from u within t steps
  - ψ(u,v,1) is efficiently computable

- For t>1, ψ(u,v,t)=(∃m)[ψ(u,m,t/2)∧ψ(m,v,t/2)]
  - m is another configuration
  - so ψ(u,v,t)=(∃m)(∀(a,b)∈{(u,m),(m,v)})[ψ(a,b,t/2)]

- *So the depth of the recursion: log $2^{O(f(n))}$=O(f(n)), and generating each configuration m takes O(f(n)) time, so the total time is $O(f^2(n))$*



- Claim 2. $\mathrm{SPACE}(f(n)) \subseteq \mathrm{ATIME}(f^2(n))$
  - *proof similar to the PSPACE-completeness of TQBF*



Main theorems

Theorem 10.21

a) For $f(n) \geq n$ we have $\mathbf{ATIME}(f(n)) \subseteq \mathbf{SPACE}(f(n)) \subseteq \mathbf{ATIME}(f^2(n))$

b) For $f(n) \geq \log n$ we have $\mathbf{ASPACE}(f(n)) = \mathbf{TIME}(2^{O(f(n))})$



Main theorems

Theorem 10.21

a) For $f(n) \geq n$ we have $\mathbf{ATIME}(f(n)) \subseteq \mathbf{SPACE}(f(n)) \subseteq \mathbf{ATIME}(f^2(n))$

b) For $f(n) \geq \log n$ we have $\mathbf{ASPACE}(f(n)) = \mathbf{TIME}(2^{O(f(n))})$

Corollary

$\mathbf{AL} = ?$

$\mathbf{AP} = ?$

$\mathbf{APSPACE} = ?$



Main theorems

Theorem 10.21

a) For $f(n) \geq n$ we have $\mathbf{ATIME}(f(n)) \subseteq \mathbf{SPACE}(f(n)) \subseteq \mathbf{ATIME}(f^2(n))$

b) For $f(n) \geq \log n$ we have $\mathbf{ASPACE}(f(n)) = \mathbf{TIME}(2^{O(f(n))})$

Corollary

$\mathbf{AL} = \mathbf{P}$

$\mathbf{AP} = \mathbf{PSPACE}$

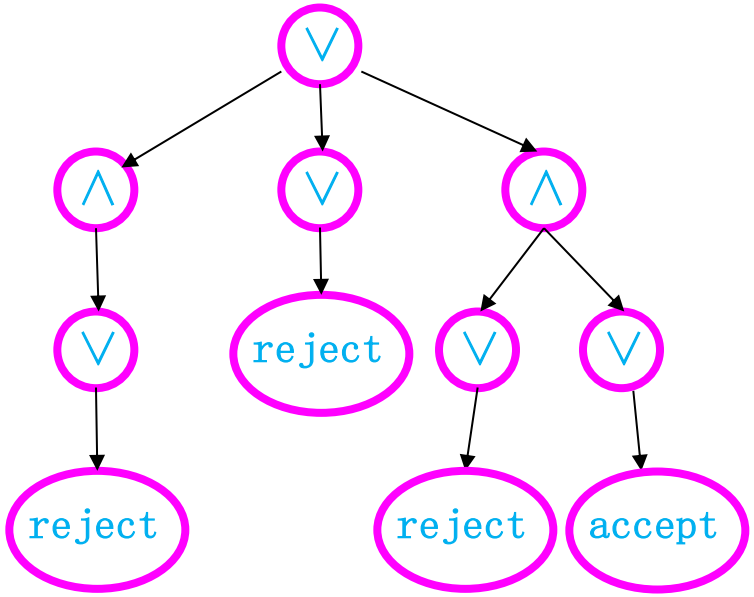
$\mathbf{APSPACE} = \mathbf{EXPTIME}$



- Claim 3. For $f(n) \geq \log n$, $ASPACE(f(n)) = TIME(2^{O(f(n))})$
- Proof:
  - $ASPACE(f(n)) \subseteq TIME(2^{O(f(n))})$

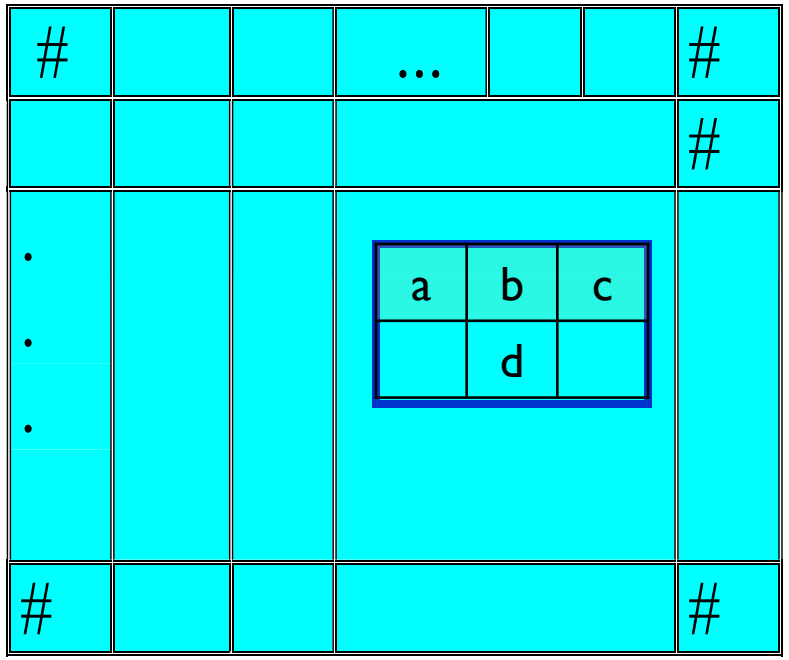


- Claim 3. For $f(n) \geq \log n$, $ASPACE(f(n)) = TIME(2^{O(f(n))})$
- Proof:
  - $ASPACE(f(n)) \subseteq TIME(2^{O(f(n))})$
  - Easy to see since we can enumerate all the possible configurations
  - Since the space is O(f(n)), the number of all configurations is $2^{O(f(n))}$
  - Construct the entire computation tree





- For $f(n) \geq \log n$, $ASPACE(f(n)) = TIME(2^{O(f(n))})$
- Proof:
  - $TIME(2^{O(f(n))}) \subseteq ASPACE(f(n))$
  - For a machine takes $2^{O(f(n))}$ time, consider the table for computation history:



- Each cell d only depends on a,b,c
- Guess all possible a,b,c
- Verify the guesses universally

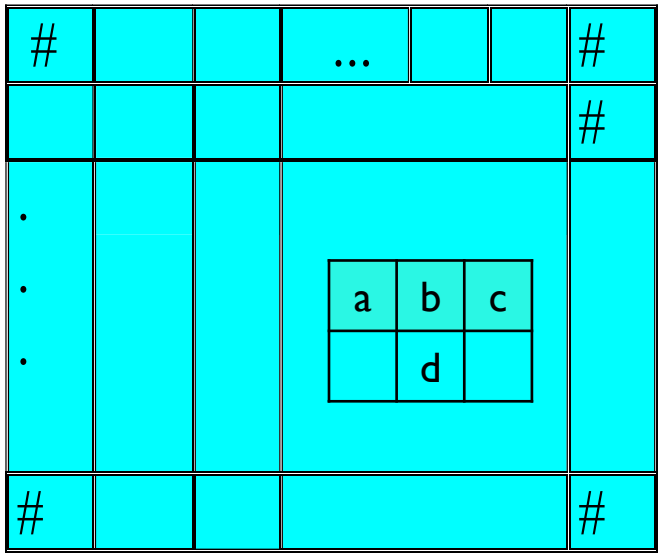
"In this model, a single symbol may represent both the state of the machine and the content of the tape cell under the head"



The alternating TM

- Let C[i,j] denote the j-th cell in time i in the configuration table

```
Guess(i, j, d)          \\Guess the C[i,j]=d
{
        For all (a,b,c) that can yield d
        {
                If all of
                  Guess(i-1,j-1, a),
                  Guess(i-1, j, b),
                  Guess(i-1, j+1, c)
                are true, return true;
        }
}
```

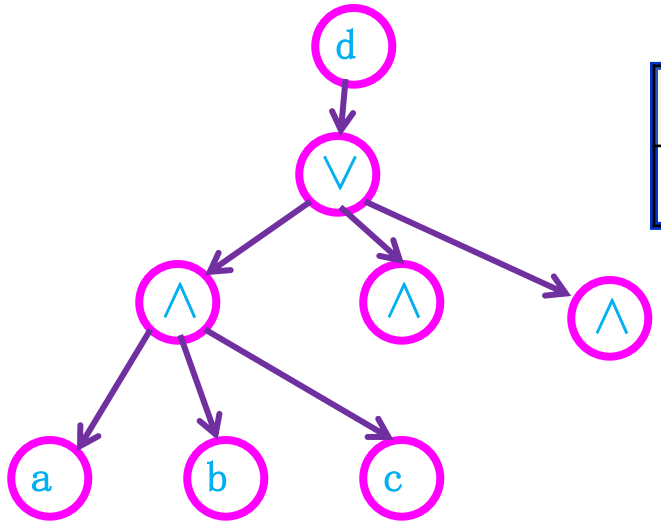


Finally, we can check Guess(n, j, q_{accept}) for all cell j in the final time n



- For $f(n) \geq \log n$, $ASPACE(f(n)) = TIME(2^{O(f(n))})$.
- Proof:
  - $TIME(2^{O(f(n))}) \subseteq ASPACE(f(n))$
  - For a machine takes $2^{O(f(n))}$ time, consider the table for computation history:

•Each cell d only depends on a,b,c
•Guess all possible a,b,c that yields d
•Verify the guesses universally



a	b	c
	d	

  - We only need to store the pointer, so the space is $\log 2^{O(f(n))} = O(f(n))$



Complexity about games?

- We have known TQBF is PSPACE-complete
  - equivalent to "formula game"
  - In a Boolean formula ψ,

    A selects the value of x_1,

    B selects the value of x_2,

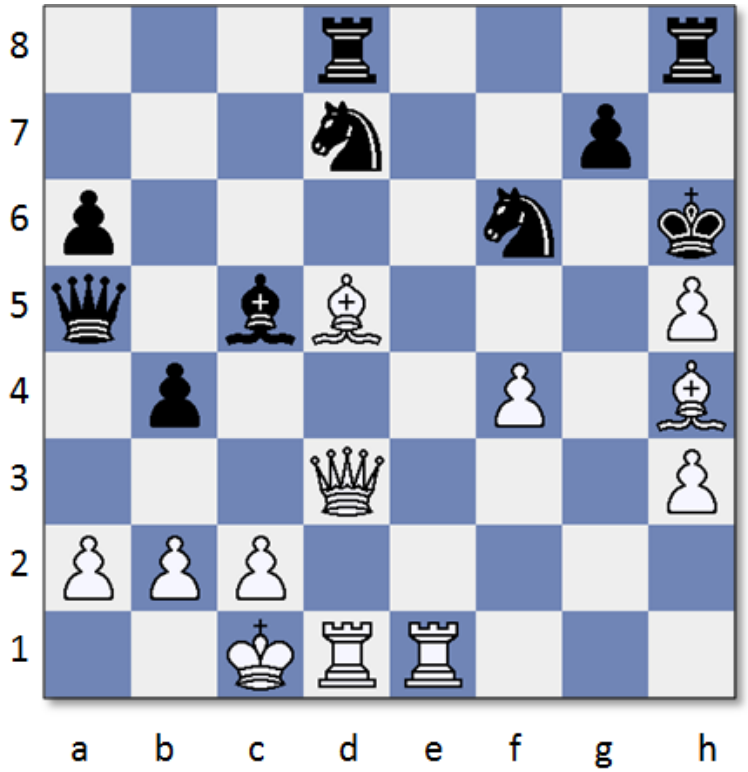
    …,

    A wins if ψ=false, otherwise B wins



Complexity about games?

- We have known TQBF is PSPACE-complete
  - equivalent to "formula game"
- What about common games? like chess:





Complexity for "generalized chess"

- We can play chess on an $n \times n$ board, with more pieces
- *[Fraenkel & Lichtenstein 1981]*

  $n \times n$ chess is EXPTIME-Complete

- Why?





Complexity for "generalized chess"

- We can play chess on an $n \times n$ board, with more pieces.
- *[Fraenkel & Lichtenstein 1981]*

  $n \times n$ chess is EXPTIME-Complete

  - The game cannot end in polynomial time! So it's not similar to the formula game
  - EXPTIME-Complete means it's not in P, by the time hierarchy theorem



- *[Stockmeyer & Chandra 1979]* "Provably difficult combinatorial games"
  - Variants of Boolean formula games which are closer to board games
  - Show that they are EXPTIME-Complete



- *[Stockmeyer & Chandra 1979]* "Provably difficult combinatorial games"
  - Variants of Boolean formula games which are closer to board games
  - Show that they are EXPTIME-Complete

- Game 1:
  - Given a Boolean formula $\psi(x_1, \ldots, x_m, y_1, \ldots, y_m, t)$
  - Player 1: set t=1 and set $x_1, \ldots, x_m$ to any values
  - Player 2: set t=0 and set $y_1, \ldots, y_m$ to any values
  - Player 1 moves first. No pass
  - A player loses if ψ is false after its move



- *[Stockmeyer & Chandra 1979]* "Provably difficult combinatorial games"
  - Variants of Boolean formula games which are closer to board games
  - Show that they are EXPTIME-Complete

- Game 1:
  - Given a Boolean formula $\psi(x_1,\ldots,x_m, y_1,\ldots,y_m, t)$, $(\psi(X,Y,t))$
  - Player 1: set t=1 and set $x_1,\ldots,x_m$ (X) to any values
  - Player 2: set t=0 and set $y_1,\ldots,y_m$ (Y) to any values
  - Player 1 moves first. No pass
  - A player loses if ψ is false after its move
  - Let Y_0 be the initial value for Y



Game 1 is EXPTIME-Complete

- It's in EXPTIME
  - Not hard to see.
  - we can enumerate all possible values of $\psi(x_1,\ldots,x_m,y_1,\ldots,y_m,t)$
- Why is it EXPTIME-hard?

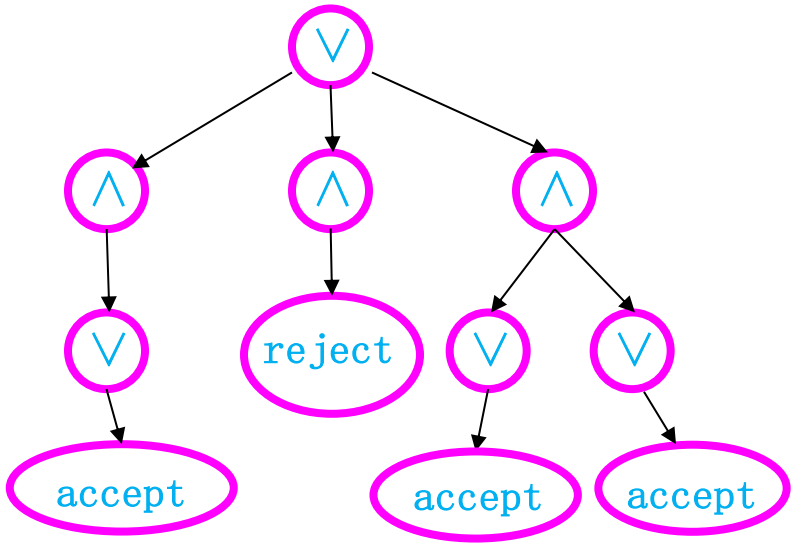
Game 1:

Given a Boolean formula $\psi(x_1,\ldots,x_m,y_1,\ldots,y_m,t)$
Player 1: set t=1 and set $x_1,\ldots,x_m$ to any values
Player 2: set t=0 and set $y_1,\ldots,y_m$ to any values
Player 1 moves first. No pass
Y_0 is the initial value for Y
A player loses if ψ is false after its move



Game 1 is "APSPACE-Complete"

- Recall that for $f(n) \geq \log n$, we have $ASPACE(f(n)) = TIME(2^{O(f(n))})$
- Suppose M is an alternating TM in polynomial space
  - Assume w.l.o.g.:
  1. the initial state is existential
  2. existential states yield universal states and universal states yield existential states
  3. accepting states are universal and rejecting states are existential





Game 1 is "APSPACE-Complete"

- Recall that for $f(n) \geq \log n$, we have $ASPACE(f(n)) = TIME(2^{O(f(n))})$
- Suppose M is an alternating TM in polynomial space
  - Assume w.l.o.g.:
  1. the initial state is existential
  2. existential states yield universal states and universal states yield existential states
  3. accepting states are universal and rejecting states are existential
  - Consider the configurations of M with length polynomial in n
  - Define NEXT(C,D) where C, D are configurations of M:
  1. When $C \vdash_M D$, NEXT(C,D)=1
  2. If C is an accepting/rejecting configuration, NEXT(C,D)=0 for all D



Game 1 is "APSPACE-Complete"

- Suppose M is an alternating TM in polynomial space
  - Consider the configurations of M with length polynomial in n
  - Define NEXT(C,D) where C, D are configurations of M:
  1. When $C \vdash_M D$, NEXT(C,D)=1
  2. If C is an accepting/rejecting configuration, NEXT(C,D)=0 for all D
- For the Game 1:
  - the Boolean formula $\psi = (\sim t \vee NEXT(Y, X)) \wedge (t \vee NEXT(X, Y))$
  - Player 1: set t=1 and set X
  - Player 2: set t=0 and set Y



Game 1 is "APSPACE-Complete"

- Suppose M is an alternating TM in polynomial space.
  - Consider the configurations of M with length polynomial in n.
  - Define NEXT(C,D) where C, D are configurations of M:
  1. When $C \vdash_M D$, NEXT(C,D)=1
  2. If C is an accepting/rejecting configuration, NEXT(C,D)=0 for all D
- For the Game 1:
  - the Boolean formula $\psi = (\sim t \vee NEXT(Y, X)) \wedge (t \vee NEXT(X, Y))$
  - Player 1: set t=1 and set X *(ψ=NEXT(Y, X))*
  - Player 2: set t=0 and set Y *(ψ=NEXT(X,Y))*
  - Player 1 moves first. No pass
  - Y_0 is the initial value for Y
  - A player loses if ψ is false after its move

Every player must select a next configuration, and try to make it a halting configuration



Reduction

For the Game 1:

the Boolean formula $\psi=(\sim t \vee NEXT(Y, X)) \wedge (t \vee NEXT(X,Y))$

Player 1: set t=1 and set X

Player 2: set t=0 and set Y

Player 1 moves first. No pass

Y_0 is the initially value for Y

A player loses if ψ is false after its move

- If Y_0 is the initial configuration of M on input w,
  Player 1 will win iff M accepts w



Reduction

the Boolean formula $\psi=(\sim t \vee NEXT(Y, X)) \wedge (t \wedge NEXT(X, Y))$

Assume:

1. the initial state is existential
2. existential states yield universal states and universal states yield existential states
3. accepting states are universal and rejecting states are existential

- Player 1 is in existential states and player 2 is in universal states
- So Player 1 selects universal state, and player 2 selects existential state
- Every player want to select a halting configuration; player 1 would select accepting state, and player 2 would select rejecting state

- Thus, Player 1 will win iff M accepts w



So Game 1 is EXPTIME-complete

For the Game 1:

the Boolean formula $\psi=(\sim t \vee NEXT(Y, X)) \wedge (t \vee NEXT(X,Y))$

Player 1: set t=1 and set X

Player 2: set t=0 and set Y

Player 1 moves first. No pass

Y_0 is the initial value for Y

A player loses if ψ is false after its move



- We can reduce Game 1 to Chess or Chinese chess





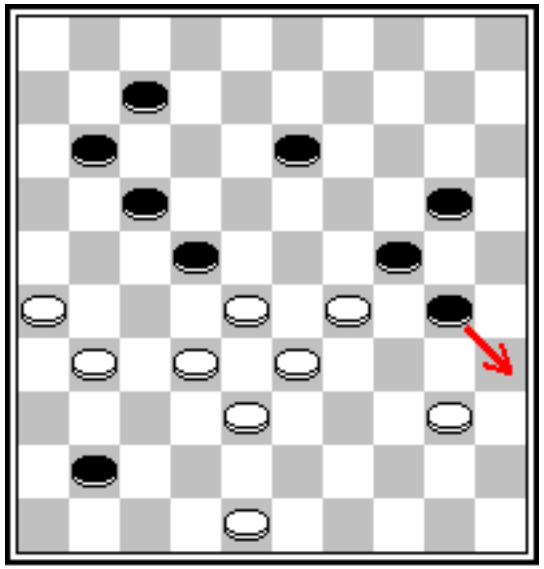


- What about checkers (跳棋)?





What about checkers (跳棋)?



Example

•Only the dark squares of the checkered board are used.
•A move consists of moving a piece diagonally to an adjacent unoccupied square. If the adjacent square contains an opponent's piece, and the square immediately beyond it is vacant, the piece may be captured (and removed from the game) by jumping over it.
•Uncrowned pieces (*men) move one step diagonally forward*
•When a man reaches the *crownhead, it becomes a king.*
•Kings (sometimes called *flying kings) can move backward, and can move any distance along unblocked diagonals*



- We can reduce Game 1 to other games
- Also EXPTIME-Complete
  - "N BY N CHECKERS IS EXPTIME COMPLETE" *[J. M. ROBSON 1984]*

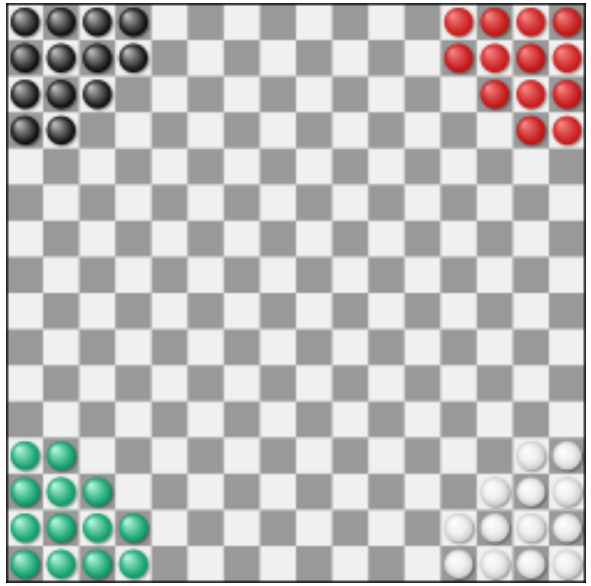




Halma and Chinese Checkers

Halma

- On each turn, a player either moves a single piece to an adjacent open square, or jumps over one or more pieces in sequence.



Chinese checkers

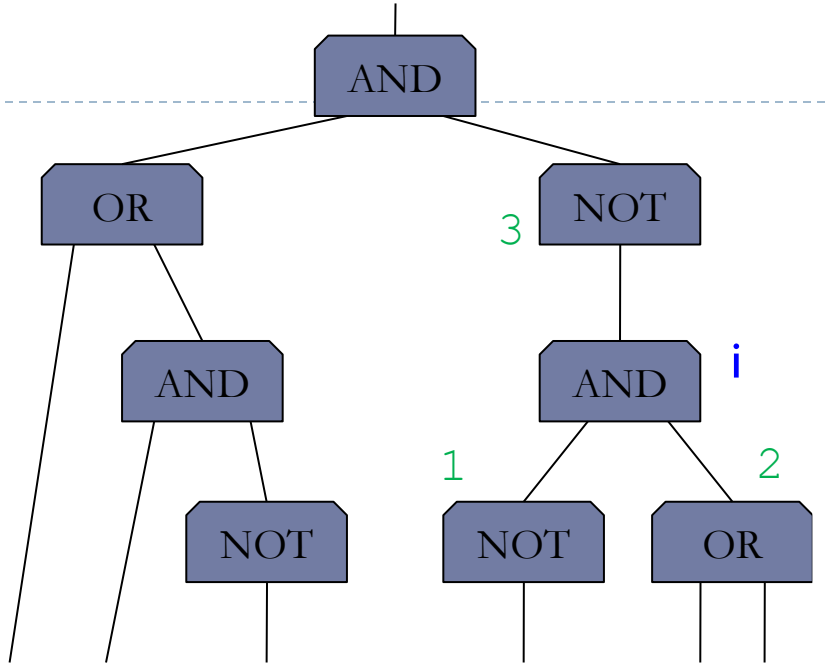
- *Chinese checkers* was invented in Germany in 1892 as "**Sternhalma**" *("Stern" means star)*
- The name "Chinese Checkers" originated in the United States as a marketing scheme by Bill and Jack Pressman in 1928.



Chinese Checker (not originated in China)

Succinct circuit SAT

- circuit SAT: given a circuit of length n



- Succinct circuit SAT:
  - given i and k, a circuit outputs the index and type of the k-th adjacent gate of the i-th gate
  - input/output of length n, polynomial circuit size
  - can encode SAT of length $2^{O(n)}$

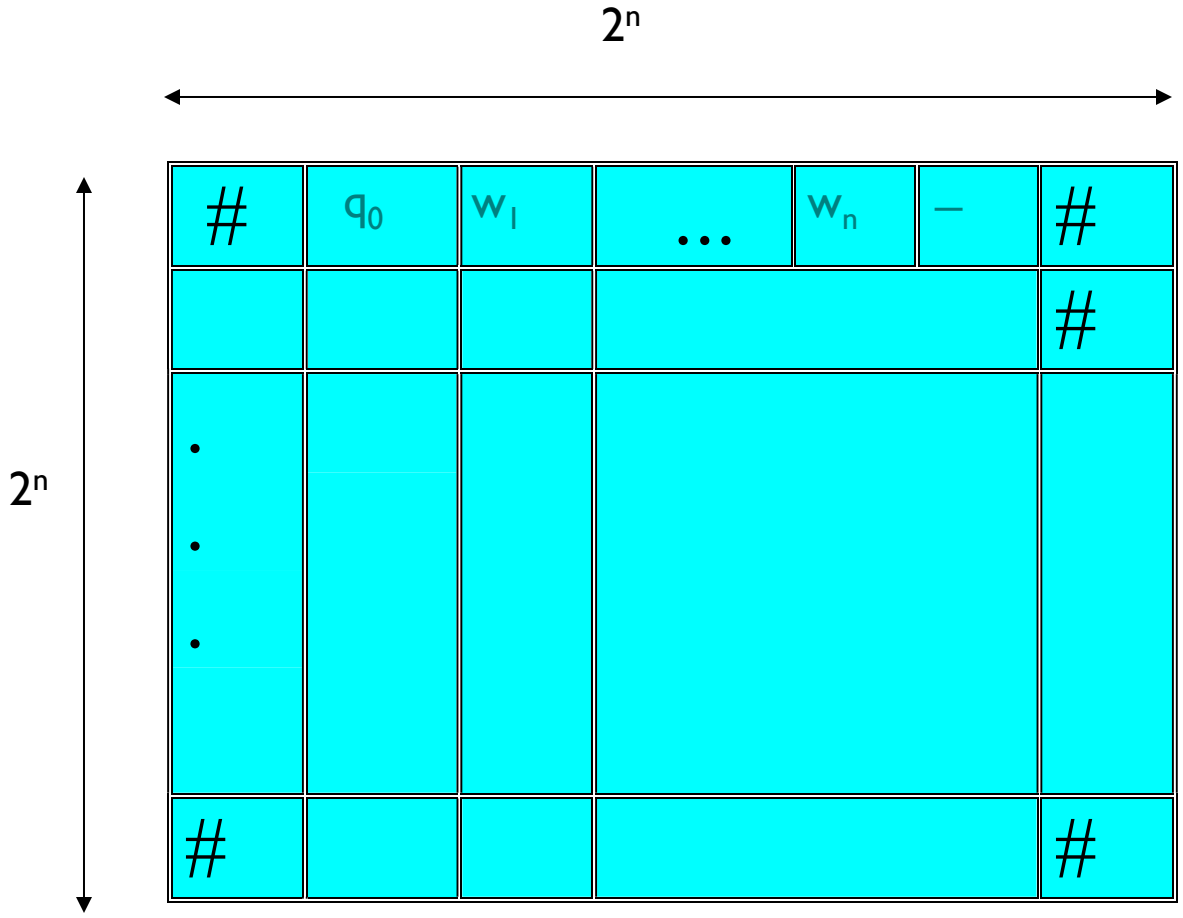
Succinct circuit SAT is in NEXP



Succinct circuit SAT is NEXP-complete

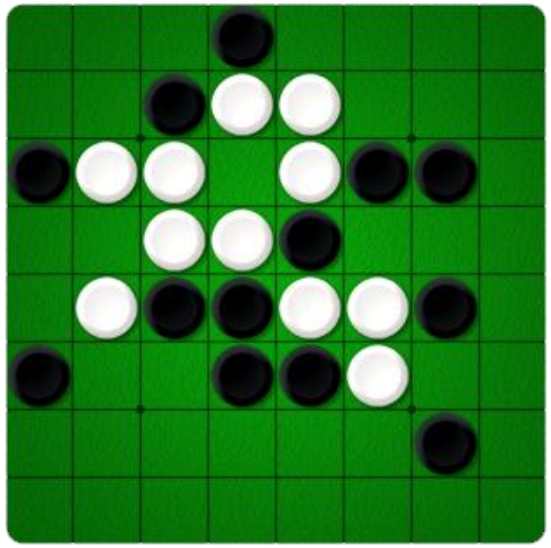
- As in the proof of NP-completeness of SAT
- We can use a poly-size circuit to generate that formula:

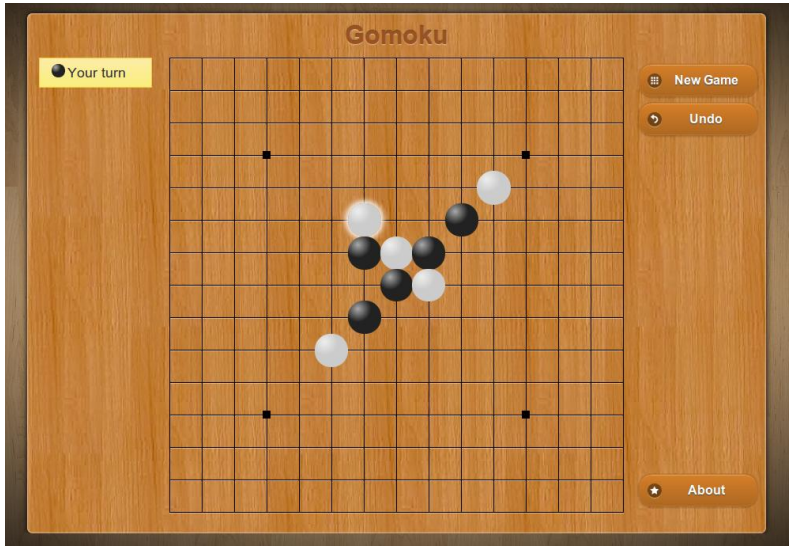
$$\varphi_{M,w} = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$





- What about reversi and Gomoku (five in a row)?



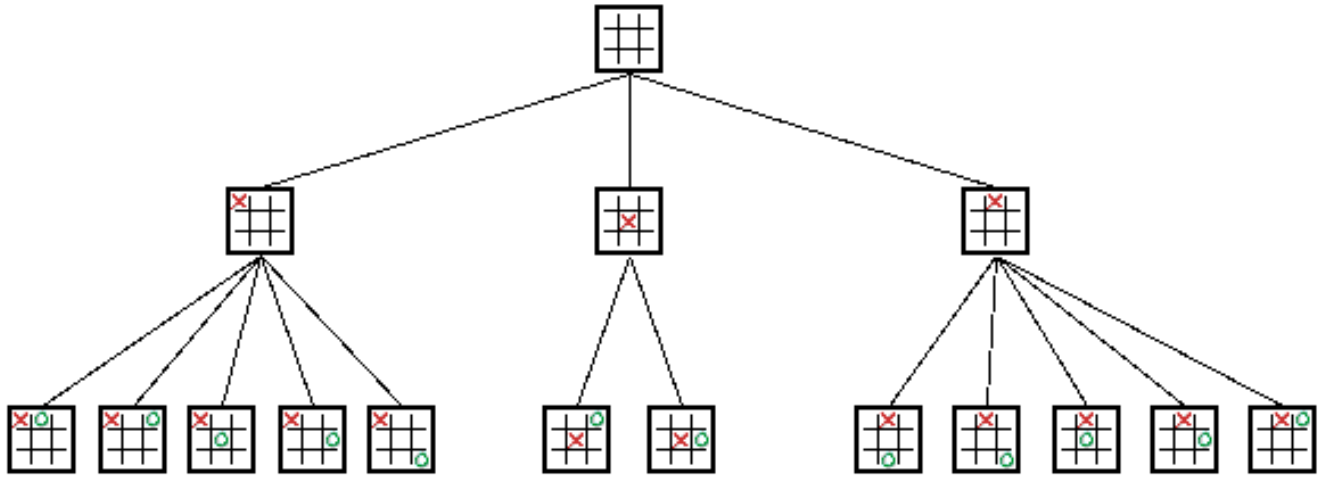




- What about reversi and Gomoku (five in a row)?
  - Can be completed in polynomial time, so the recursive search of the game tree takes polynomial space
  - PSPACE-Complete:

    S. Iwata and T. Kasai, "The Othello game on an n*n board is PSPACE-complete", *Theor. Comp. Sci. 123 (1994) 329-340.*

    Stefan Reisch (1980). "Gobang ist PSPACE-vollständig (Gomoku is PSPACE-complete)". *Acta Informatica* ***13: 59–66.***



- What about GO(圍棋)?
  - with Japanese ko rule, it is proved to be EXPTIME-Complete

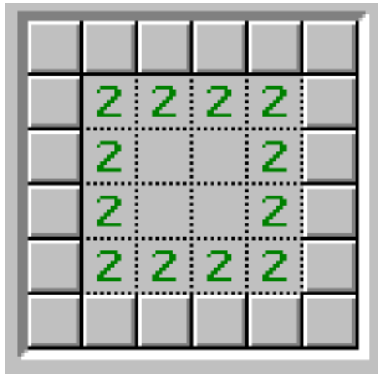
    J. M. Robson. *The complexity of Go.* In Proceedings of the IFIP 9th World Computer Congress on Information Processing, pages 413–417, 1983.
  - some rules can guarantee to end in polynomial time, so they are PSPACE-Complete
  - with superko rule it remains an open question

    See *Games, Puzzles, and Computation*, Robert Hearn's Ph.D. thesis.



Minesweeper (NP-complete)

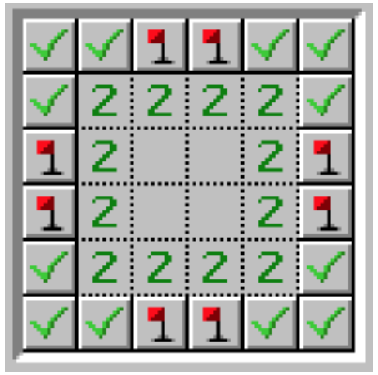
- The number indicating how many mines surround that tile
- Try to locate the mines
- Example:
  - can you play this without a guess?





Minesweeper

- Solution:

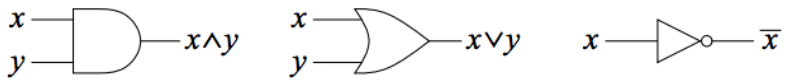


- A Minesweeper board is *consistent* if there is at least one possible configuration of mines and safe squares that does not result in a contradiction
  - finding solutions ⟺ checking consistency given a tile fixed

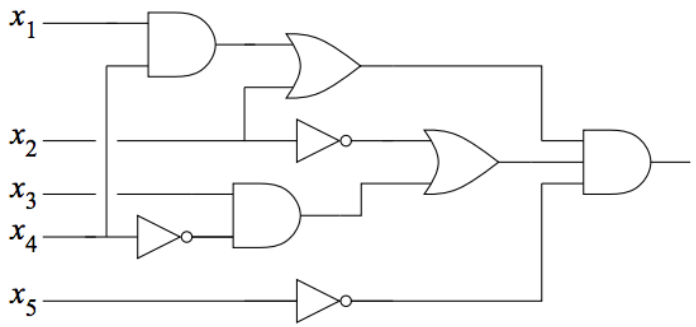


Minesweeper consistency is NP-complete

- in NP: trivial
- reduces circuit satisfiability to Minesweeper
  - we need to implement wires and gates:



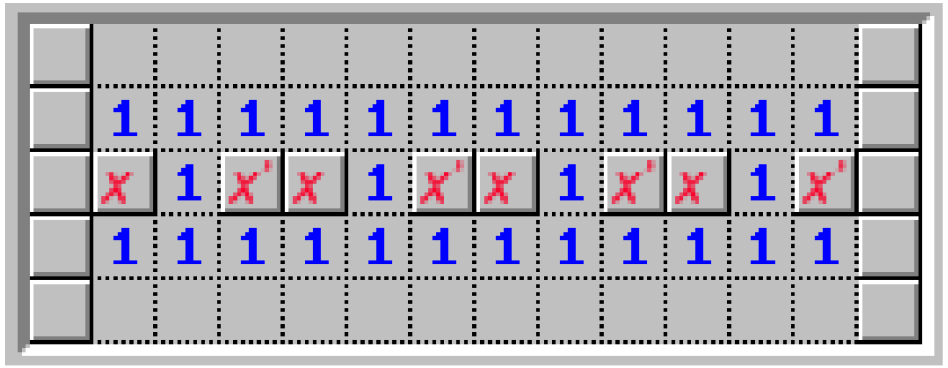
An AND gate, an OR gate, and a NOT gate.



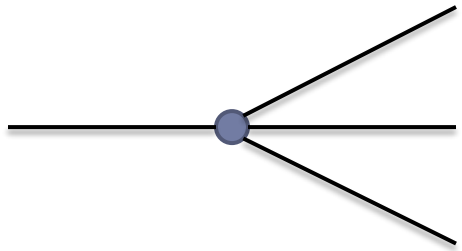
A boolean circuit. Inputs enter from the left, and the output leaves to the right.



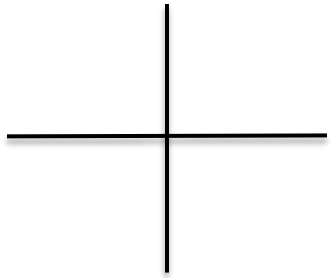
Wire and NOT gate



- what about:



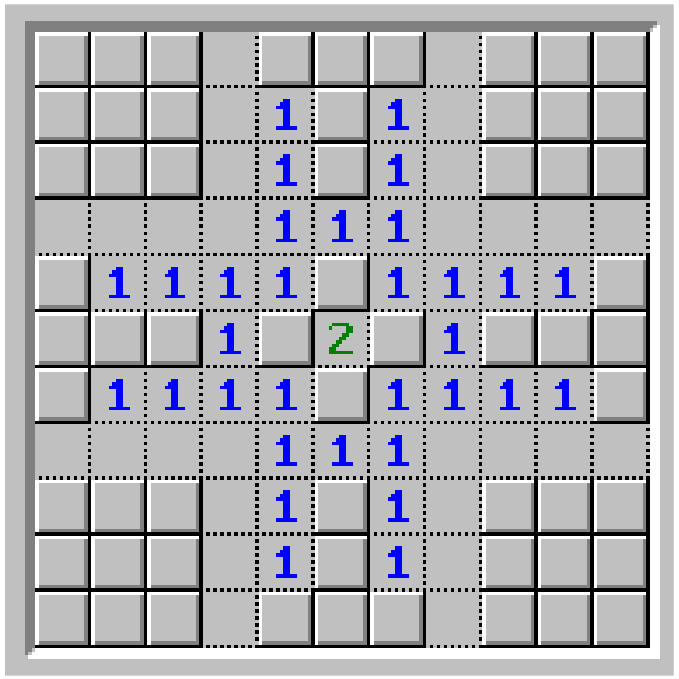
splitter



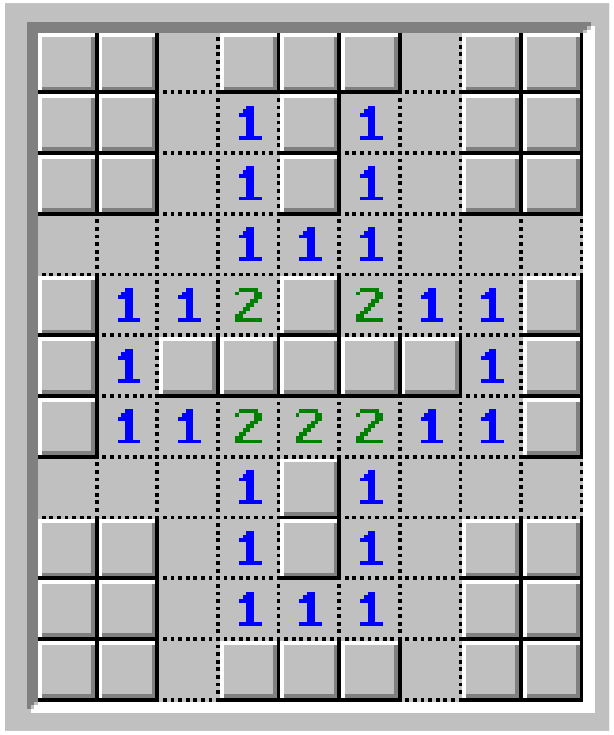
crossover



- So the circuit doesn't need to be planar



splitter

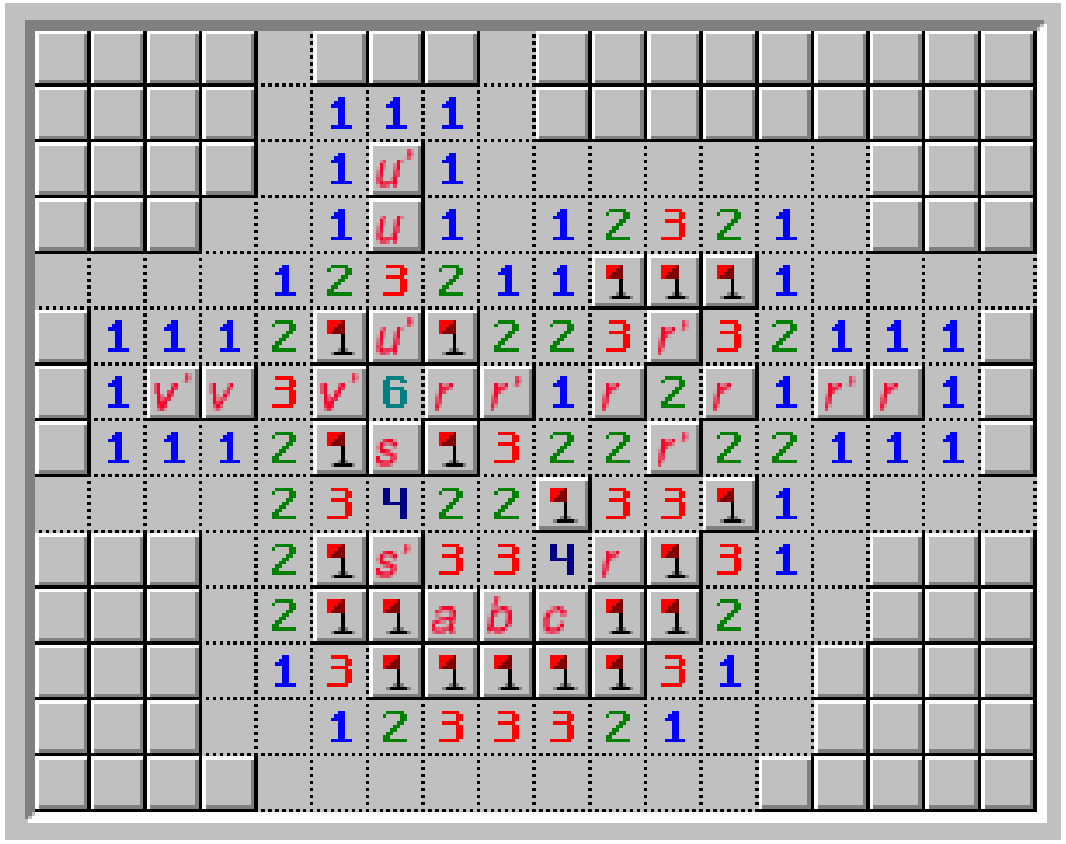


crossover



OR gate

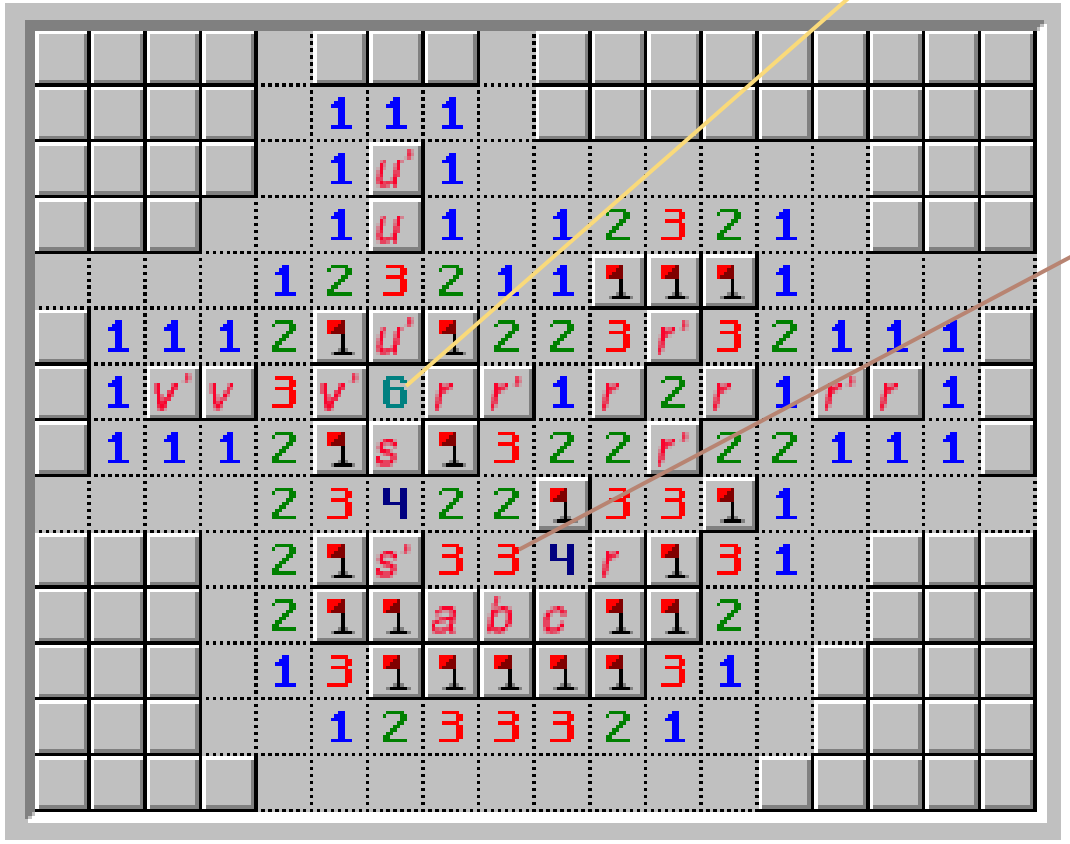
- r=uVv


1 1 1
1 u' 1
1 u 1 1 2 3 2 1
1 2 3 2 1 1 1 1 1 1
1 1 1 2 1 u' 1 2 2 3 r' 3 2 1 1 1
1 v' v 3 v' 6 r r' 1 r 2 r 1 r' r 1
1 1 1 2 1 s 1 3 2 2 r' 2 2 1 1 1
2 3 4 2 2 1 3 3 1 1
2 1 s' 3 3 4 r 1 3 1
2 1 1 a b c 1 1 2
1 3 1 1 1 1 1 3 1
1 2 3 3 3 2 1

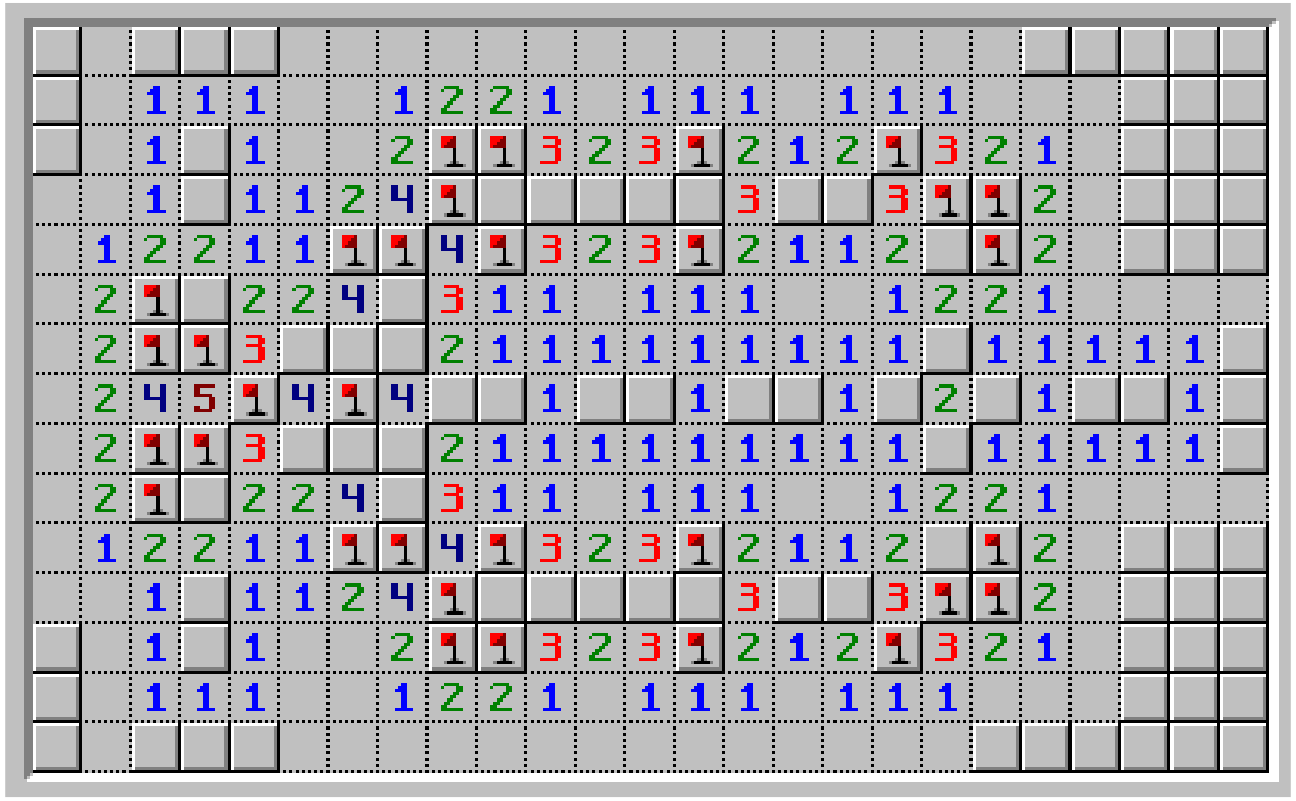


OR gate

- r=uVv

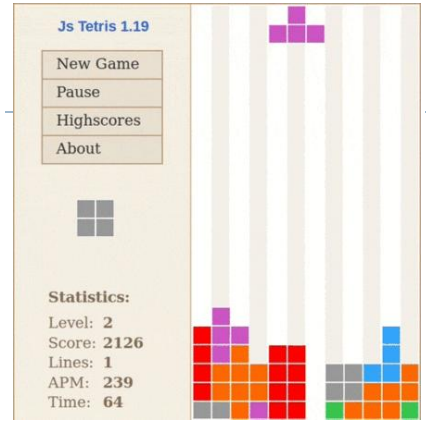


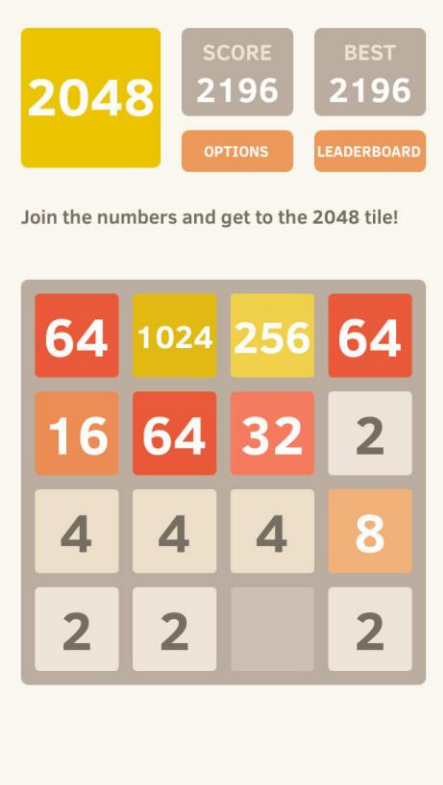
AND gate



NP-hard games

- Tetris (俄罗斯方块)
- Cross Sums
- Minesweeper
- 2048
- *[Aloupis, Demaine, Guo 2012]* "Classic Nintendo Games are (NP-)hard"
  - Pokémon
  - Mario
  - Donkey Kong
  - Legend of Zelda
  - Metroid
- …





What about Wordle?





Wordle is NP-HARD

Daniel Lokshtanov[1] and Bernardo Subercaseaux[2]

[1]University of California Santa Barbara
[2]Carnegie Mellon University, Pittsburgh

April 1, 2022

Abstract

Wordle is a single player word-guessing game where the goal is to discover a secret word w that has been chosen from a dictionary D. In order to discover w, the player can make at most ℓ guesses, which must also be words from D, all words in D having the same length k. After each guess, the player is notified of the positions in which their guess matches the secret word, as well as letters in the guess that appear in the secret word in a different position. We study

PROBLEM :	Wordle
INPUT :	(D, ℓ) with $D \subseteq \Sigma^k$ the dictionary, and an integer $\ell \geq 1$, the number of guesses allowed.
OUTPUT :	**Yes**, if there is a winning strategy for the guesser, and **No** otherwise.



Football elimination

Team	Points
A	6
B	4
C	4
D	2
E	1

- In a league (past), 2 points for a win, 1 point for a draw, and 0 point for a lose

- Input:
  - current league table
  - games left
  - a team T
- decide whether T can still win the champion

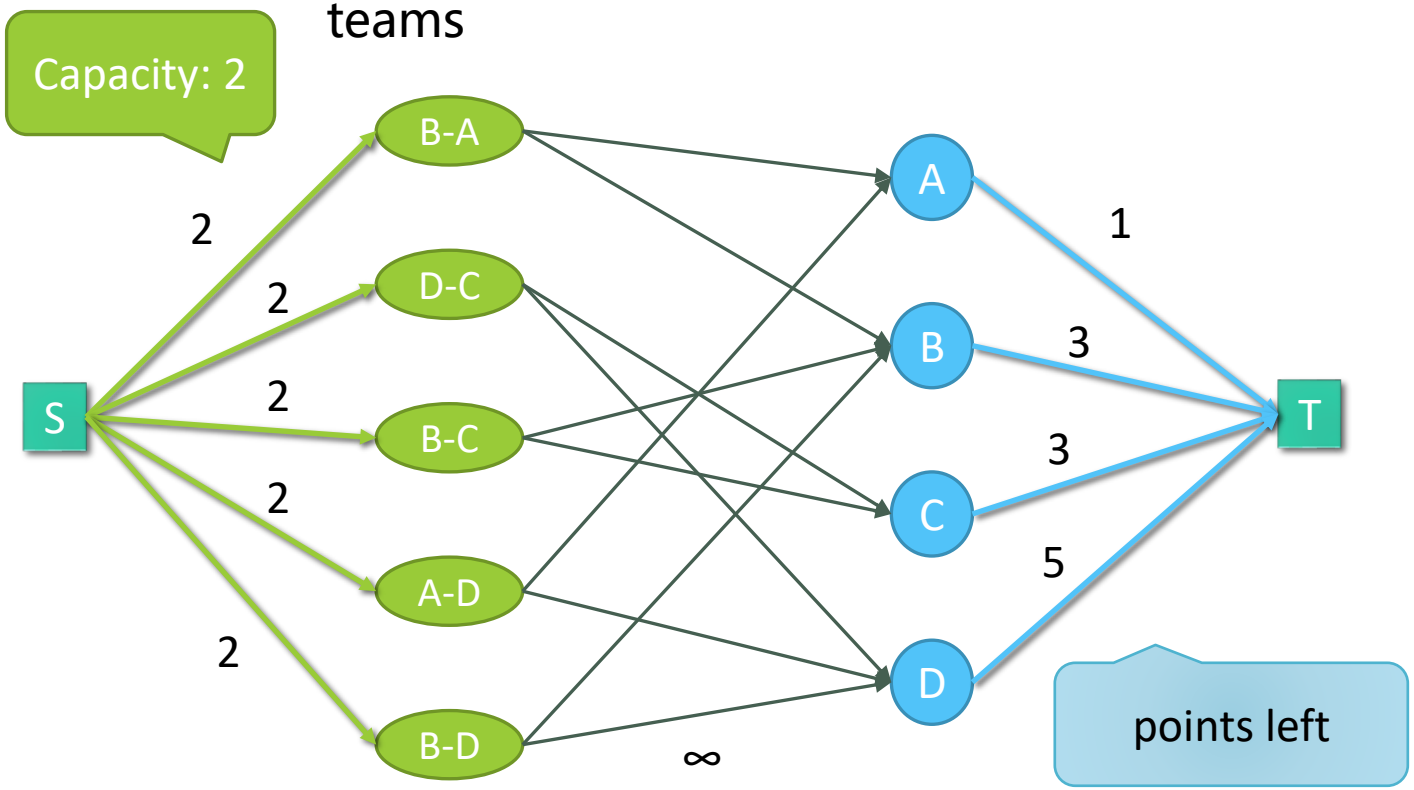
- Can be reduced to max-flow
  - thus in polynomial time

Games left
A-E
E-C
B-A
D-C
B-C
B-E
A-D
B-D

Can E still win?



- E wins all of its games, at most 7 points
- Other team cannot exceed 7 points
- Consider all games between other teams



E can still win⇔ max-flow is 10 (all edges from s are saturated)

球队	积分
A	6
B	4
C	4
D	2
E	1

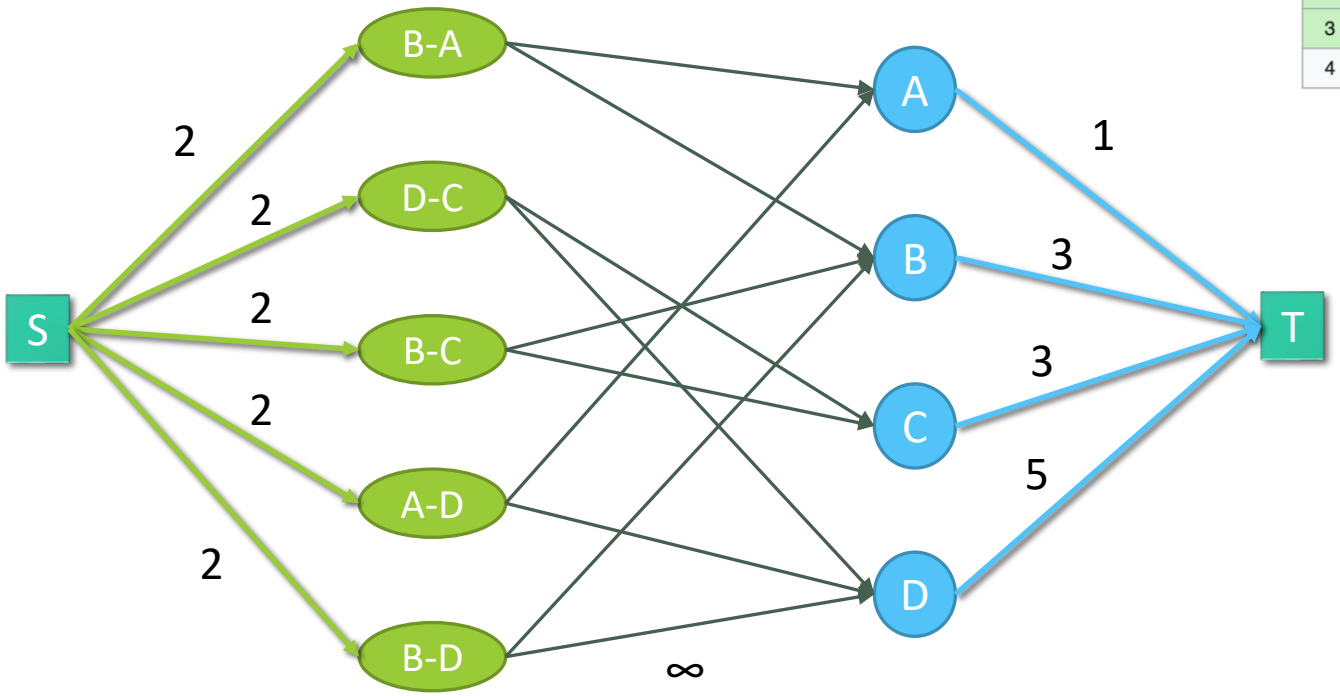
剩余赛程
A-E
E-C
B-A
D-C
B-C
B-E
A-D
B-D

E是否有夺冠可能？

- FIFA adopted 3-point rule in 1994: (England in 1981)
  - 3 points for a win, 1 point for a draw, and 0 point for a lose

1994 FIFA World Cup Group E

Pos	Team [v·t·e]	Pld	W	D	L	GF	GA	GD	Pts	Qualification
1	Mexico	3	1	1	1	3	3	0	4	Advance to knockout stage
2	Republic of Ireland	3	1	1	1	2	2	0	4	
3	Italy	3	1	1	1	2	2	0	4	
4	Norway	3	1	1	1	1	1	0	4	



This reduction doesn't work any more

Football elimination is NP-complete under 3-point rule *[Bernholt et al. 1999]*

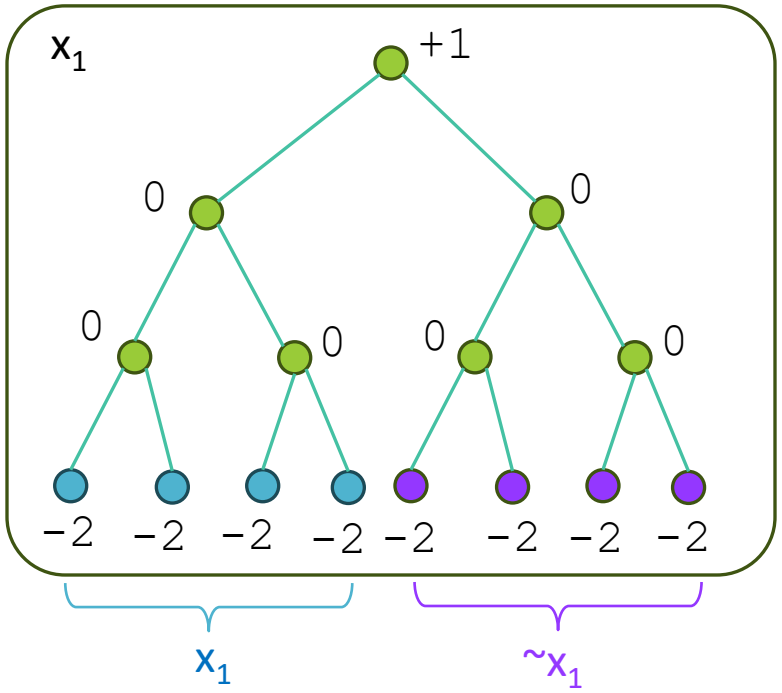
- by reducing 3-SAT to it

Reduce 3-SAT to football elimination w. 3-point rule *[Bernholt et al. 1999]*

m clauses, n variables 3CNF:

$$[(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$$

vertex: player; edge: game
label: points if all games are draws
we want all points are ≤0

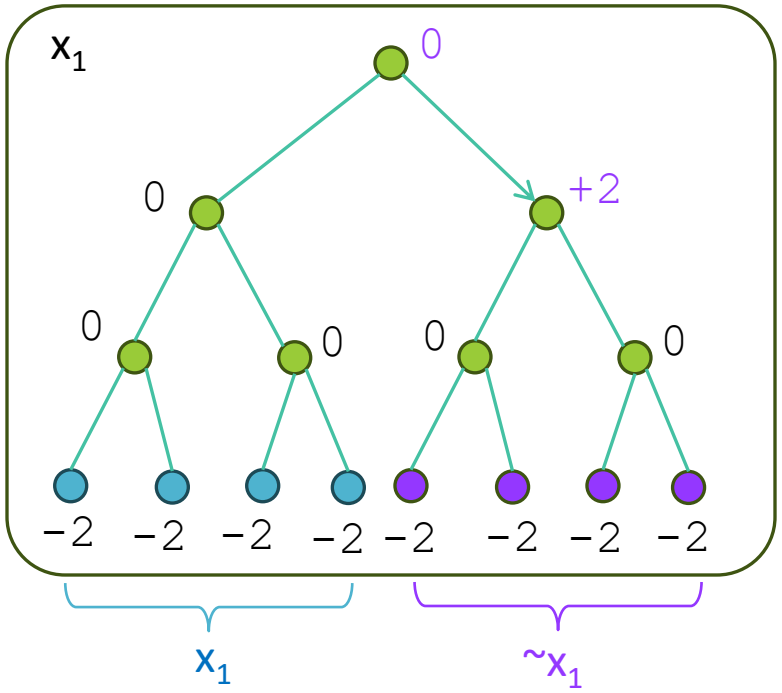


Reduce 3-SAT to football elimination w. 3-point rule *[Bernholt et al. 1999]*

m clauses, n variables 3CNF:

$$[(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$$

vertex: player; edge: game
label: points if all games are draws
we want all points are ≤0

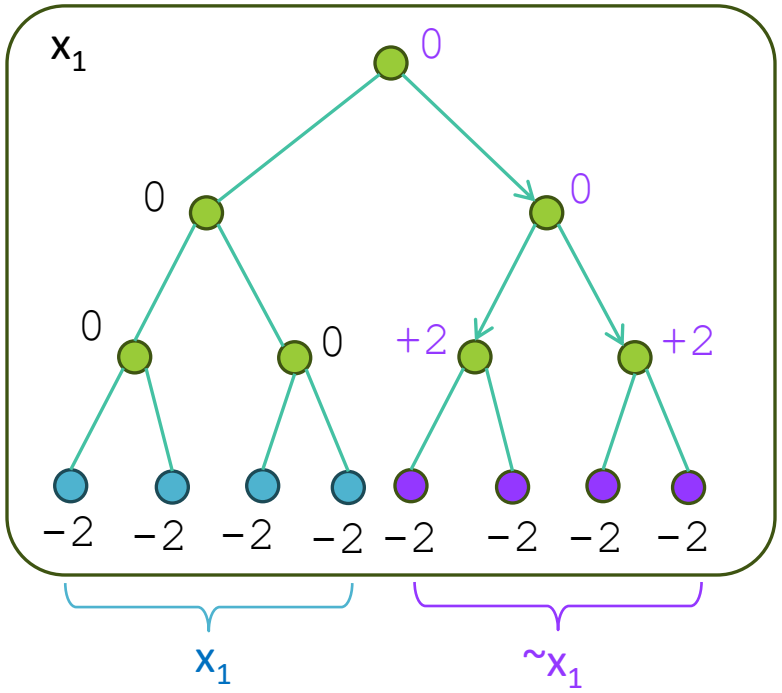


Reduce 3-SAT to football elimination w. 3-point rule *[Bernholt et al. 1999]*

m clauses, n variables 3CNF:

$$[(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$$

vertex: player; edge: game
label: points if all games are draws
we want all points are ≤0

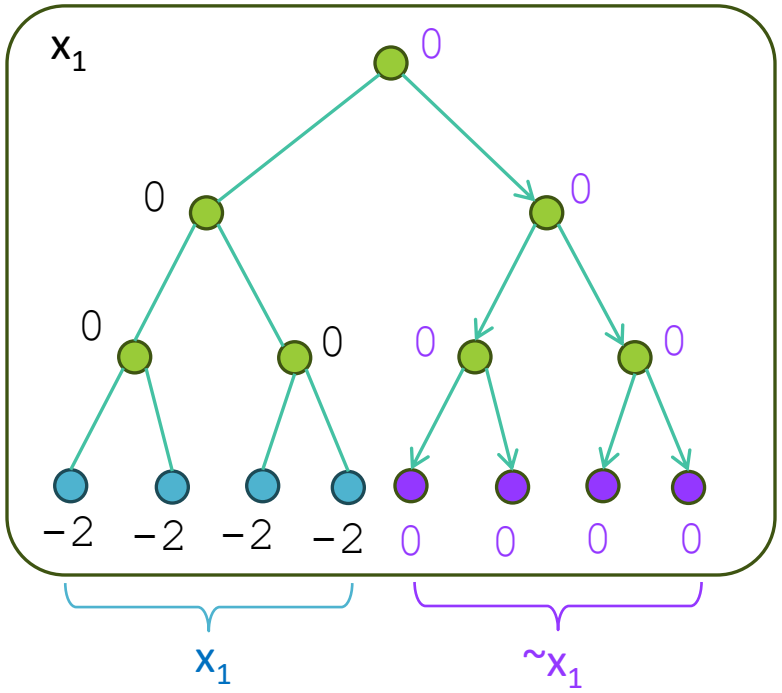


Reduce 3-SAT to football elimination w. 3-point rule *[Bernholt et al. 1999]*

m clauses, n variables 3CNF:

$$[(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$$

vertex: player; edge: game
label: points if all games are draws
we want all points are ≤0

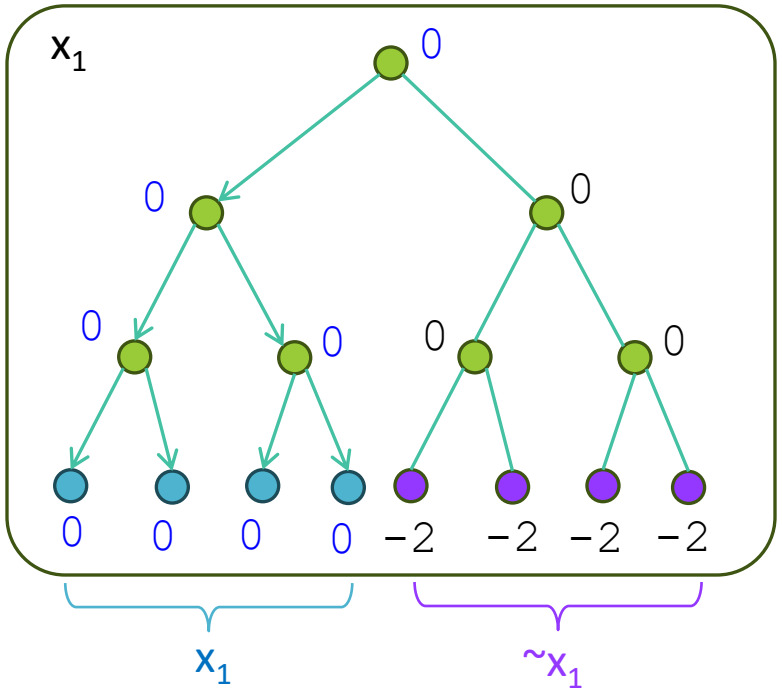


Reduce 3-SAT to football elimination w. 3-point rule *[Bernholt et al. 1999]*

m clauses, n variables 3CNF:

$$[(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$$

vertex: player; edge: game
label: points if all games are draws
we want all points are ≤0

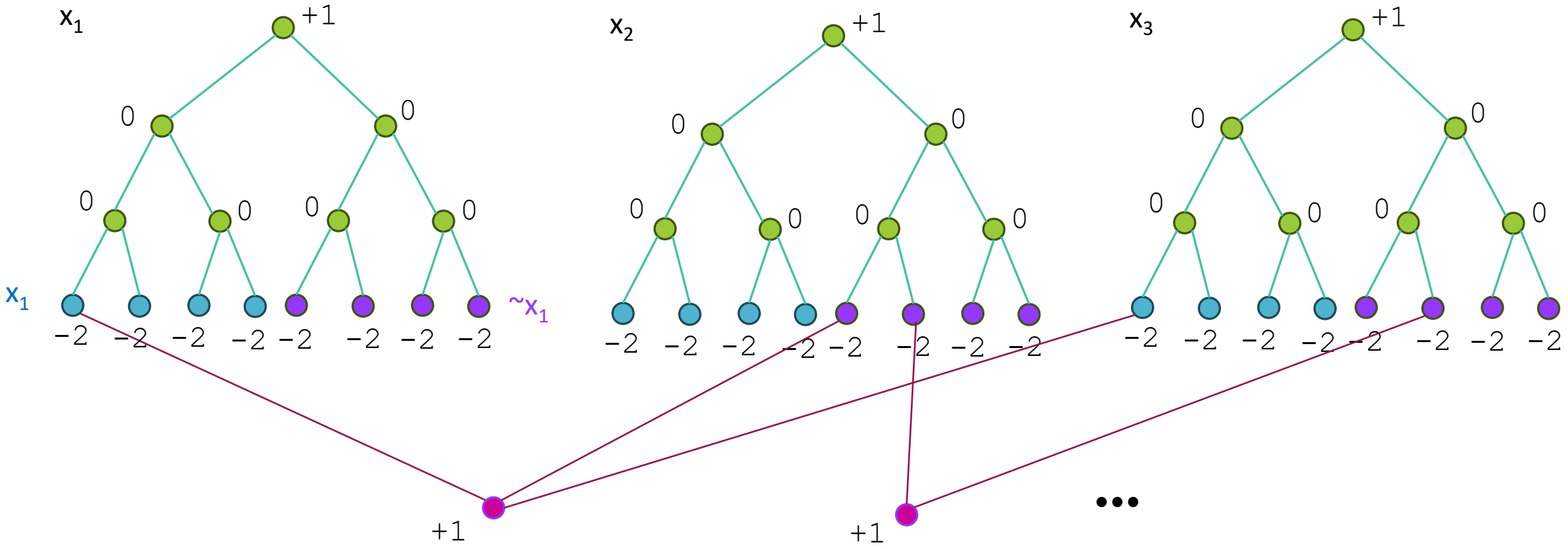


Reduce 3-SAT to football elimination w. 3-point rule *[Bernholt et al. 1999]*

m clauses, n variables 3CNF:

$[(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$

vertex: player; edge: game
label: points if all games are draws
we want all points are ≤0

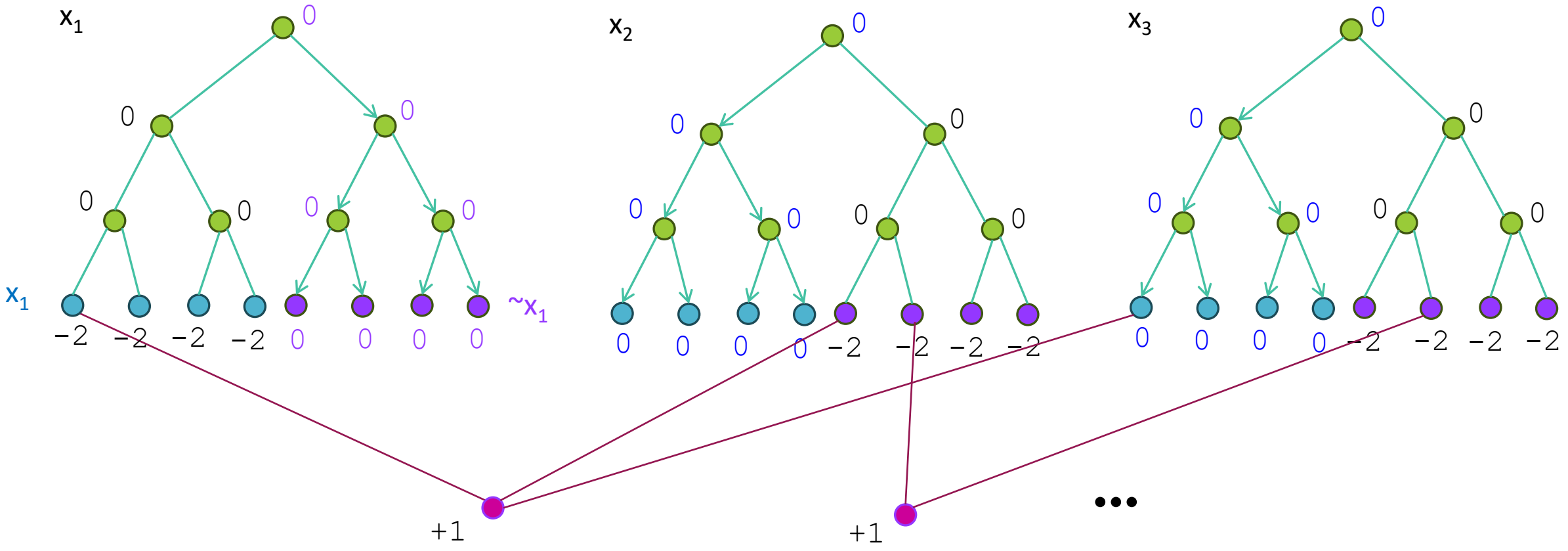


Reduce 3-SAT to football elimination w. 3-point rule *[Bernholt et al. 1999]*

m clauses, n variables 3CNF:

$[(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$

vertex: player; edge: game
label: points if all games are draws
we want all points are ≤0

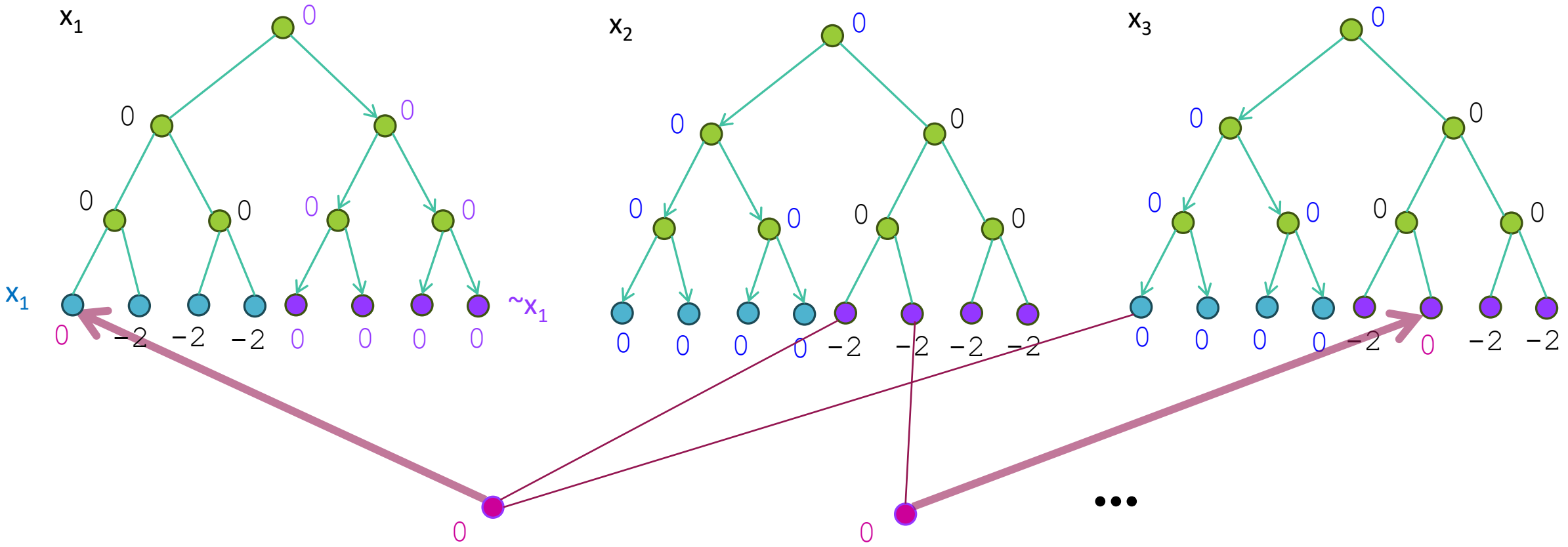


Reduce 3-SAT to football elimination w. 3-point rule *[Bernholt et al. 1999]*

m clauses, n variables 3CNF:

$[(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_2 \vee \sim x_3 \vee \ldots) \wedge \ldots]$

vertex: player; edge: game
label: points if all games are draws
we want all points are ≤0



Relativization

Diagonalization

- We have used diagonalization to prove:
  - Real number are uncountable
  - Gödel's incompleteness theorem
  - Halting problem is undecidable
  - The existence of NP intermediate problem if P≠NP
  - Time/Space Hierarchy theorem, thus L≠PSPACE, and P≠EXPTIME, which means Generalized Chess∉P

- What about P and NP?



Relativization

- Many proofs and techniques we have seen are **relativizing**:
  - they hold after replacing all TMs with oracle TMs that have access to an oracle A
  - including diagonalization
  - e.g. $\mathbf{L}^{\mathbf{A}} \subseteq \mathbf{P}^{\mathbf{A}}$ for all oracles A
  - e.g. $\mathbf{P}^{\mathbf{A}} \neq \mathbf{EXP}^{\mathbf{A}}$ for all oracles A



Relativization

- Idea: design an oracle A relative to which some statement is *false*
  - implies there can be no **relativizing proof** of that statement
  - e.g. design A for which $\mathbf{P^A = NP^A}$
- Better: also design an oracle B relative to which the statement is *true*
  - e.g. also design B for which $\mathbf{P^B \neq NP^B}$
  - implies no **relativizing proof** can resolve truth of the statement either way!



Oracles for **P** vs. **NP**

- *[Baker, Gill, and Solovay 1975]*
  - oracle A for which $\mathbf{P}^A = \mathbf{NP}^A$
  - oracle B for which $\mathbf{P}^B \neq \mathbf{NP}^B$

- conclusion: resolving

  P vs. NP

  requires a non-relativizing proof



Oracles for **P** vs. **NP**

- for $\mathbf{P}^{A} = \mathbf{NP}^{A}$ need A to be powerful
  - warning: intend to make **P** more powerful, but also make **NP** more powerful
  - e.g. A = SAT doesn't work



Oracles for **P** vs. **NP**

- for $\mathbf{P}^{A} = \mathbf{NP}^{A}$ need A to be powerful
  - warning: intend to make **P** more powerful, but also make **NP** more powerful
  - e.g. A = SAT doesn't work
  - however A = TQBF works:

$$\mathbf{PSPACE} \subseteq \mathbf{P}^{\mathbf{TQBF}} \subseteq \mathbf{NP}^{\mathbf{TQBF}} \subseteq \mathbf{NPSPACE}$$

and we know **NPSPACE = PSPACE**



Oracles for **P** vs. **NP**

Theorem: there exists an oracle B for which $\mathbf{P^B \neq NP^B}$

- Proof:
  - define a language (depending on another language B)

$$L(B) = \{1^k : \exists x \in B \text{ s.t. } |x| = k\}$$

Example: B={1,100,101,1110,1110101,1001010,…}
L={1,111, 1111,1111111,...}

  - we will find a language B such that $L(B) \in \mathbf{NP^B - P^B}$
  - easy: $L(B) \in \mathbf{NP^B}$ (no matter what B is)

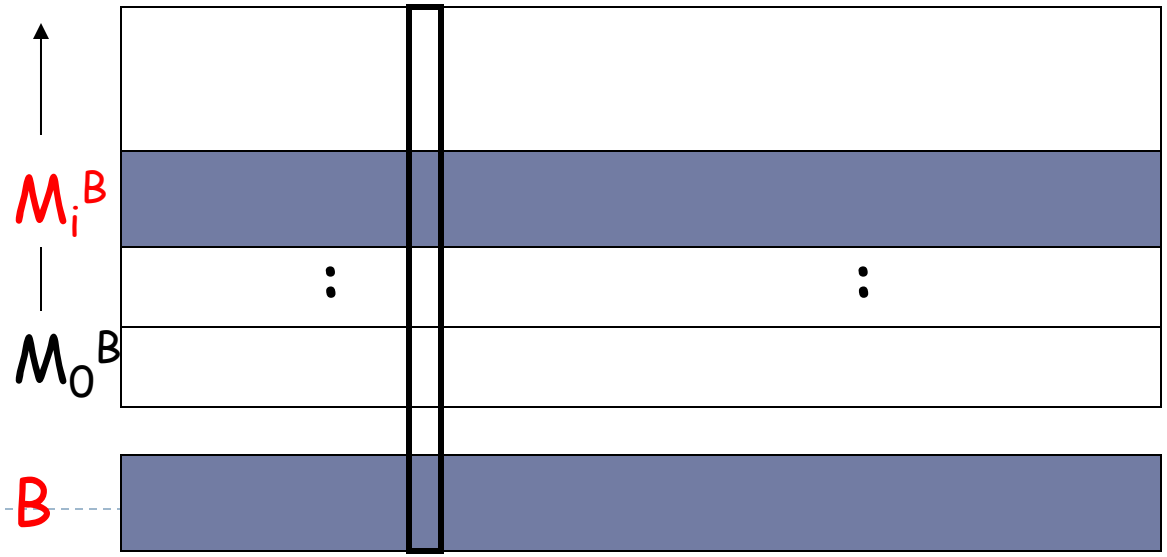
    x is a "certificate": to show $1111 \in L(B)$, gives $1110 \in B$



Oracles for **P** vs. **NP**

- Our aim: find a B such that $L(B) \notin \mathbf{P}^B$
  - design B by diagonalizing against all

    "$\mathbf{P}^B$ machines"
  - $M_0, M_1, M_2, \ldots$ is an enumeration of deterministic OTMs in n^i time
  - each machine appears infinitely often
  - We build B by simulating machine M_i one by one



Oracles for **P** vs. **NP**

We want to prove $L(B) \notin \mathbf{P}^B$

$$L(B) = \{1^k : \exists\, x \in B \text{ s.t. } |x| = k\}$$

- Proof (continued):
  - B, X= ∅
  - X: strings that are excluded from B
  - B and X are disjoint!



Oracles for **P** vs. **NP**

We want to prove $L(B) \neq M_i^B$ for all M_i (in poly time)

$$L(B) = \{1^k : \exists\ x \in B \text{ s.t. } |x| = k\}$$

- Proof (continued):
  - B, X= ∅ *(B and X are disjoint!)*

Stage i:

- select n so that n>|x| for all x∈B and x∈X currently
- simulate $M_i(1^n)$ for $n^{\log n}$ steps
- when M_i makes an oracle query q:
  - if |q| < n, answer using B
  - if |q| ≥ n, answer "no"; add q to X
- if simulated M_i accepts 1^n, add all $\{0,1\}^n$ to X *(so $1^n \notin L(B)$)*
- if simulated M_i rejects 1^n, add $\{0,1\}^n \backslash X$ to B



We want to prove that all M_i^B don't decide L(B) in poly time

$L(B) = \{1^k : \exists\ x \in B \text{ s.t. } |x| = k\}$

- Proof (continued):
  - B, X= ∅

•B and X are disjoint
•if M_i accepts 1^n, B doesn't contain any string of length n, so $1^n \notin L(B)$
•if M_i rejects 1^n, the number of strings of length n in X is $\leq n^{\log n} < 2^n$, so B contains some string of length n, so $1^n \in L(B)$

Stage i:

- select n so that n>|x| for all x∈B and x∈X currently
- simulate $M_i(1^n)$ for $n^{\log n}$ steps
- when M_i makes an oracle query q:
  - if |q| < n, answer using B
  - if |q| ≥ n, answer "no"; add q to X
- if simulated M_i accepts 1^n, add all $\{0,1\}^n$ to X
- if simulated M_i rejects 1^n, add $\{0,1\}^n \backslash X$ to B



We want to prove that all M_i^B don't decide L(B) in poly time

$L(B) = \{1^k : \exists\ x \in B \text{ s.t. } |x| = k\}$

- Proof (continued):
  - B, X= ∅

Stage i:

- select n so that n>|x| for all x∈B and x∈X currently
- simulate $M_i(1^n)$ for $n^{\log n}$ steps
- when M_i makes an oracle query q:
  - if |q| < n, answer using B
  - if |q| ≥ n, answer "no"; add q to X
- if simulated M_i accepts 1^n, add all $\{0,1\}^n$ to X
- if simulated M_i rejects 1^n, add $\{0,1\}^n \backslash X$ to B

M_i is an n^i-time TM, we can choose n such that log n>i



$L = \{1^k : \exists\, x \in B \text{ s.t. } |x| = k\}$

- Proof (continued):
  - B, X= ∅

•B and X are disjoint
•if M_i accepts 1^n, B doesn't contain any string of length n, so $1^n \notin L(B)$
•if M_i rejects 1^n, the number of strings of length n in X is $\leq n^{\log n} < 2^n$, so B contains some string of length n, so $1^n \in L(B)$

Stage i:

- select n so that $n > |x|$ for all $x \in B$ and $x \in X$ currently
- simulate $M_i(1^n)$ for $n^{\log n}$ steps
- when M_i makes an oracle query q:
  - if $|q| < n$, answer using B
  - if $|q| \geq n$, answer "no"; add q to X
- if simulated M_i accepts 1^n, add all $\{0,1\}^n$ to X
- if simulated M_i rejects 1^n, add $\{0,1\}^n \setminus X$ to B

Therefore, $L(B) \notin \text{TIME}(n^{\log n})^B$, so $L(B) \notin \mathbf{P}^B$

Oracles for **P** vs. **NP**

- **Theorem**: there exists an oracle B for which $\mathbf{P^B \neq NP^B}$
  - The proof is based on diagonalization

- **So relativizing proof doesn't show the relationship of P and NP**



Summary

- Time hierarchy theorem
- Ladner's theorem
  - if P≠NP, then there exists NP-intermediate problems
- Relativization: diagonalization cannot prove P versus NP
- Polynomial Hierarchy/Oracle TM
- EXPTIME-Completeness

